ORIGINAL PAPER

JutulDarcy.jl - a fully differentiable high-performance reservoir simulator based on automatic differentiation

Olav Møyner[1]

Received: 27 December 2024 / Accepted: 3 June 2025 / Published online: 11 July 2025

Abstract

Reservoir simulators are highly complex computer programs and are often treated as "black boxes" that act on standardized input formats. In part, this is due to the closed-source nature of commercial offerings commonly used in industry, but even when the source-code is available, modification of a fully featured simulator can be a daunting task: A reservoir simulation problem, when fully specified, can have a great number of parameters and functional relationships that are tightly coupled together in most implementations. This is an obstacle to workflows that go beyond forecasting and into optimization, parameter fitting and sensitivity analysis where the solution quantity of interest should be paired with gradients taken with respect to forces and parameters. The monolithic design of typical reservoir simulation codes cannot be separated from the technical and mathematical choices made when discretizing the governing equations: The implicitness required to handle pressure changes and strong coupling between different solution variables means that the Jacobian of the governing equations must be implemented with respect to the primary variables: An error-prone process when done manually, and highly time-consuming when considering coupled systems. Automatic differentiation (AD) is a common cure to this problem that can generate Jacobians, but often comes at a severe cost in runtime performance. A more practical issue is that high computational performance for simulation problems relevant to modern geoenergy operations necessitates implementation in a compiled language while optimization and history matching workflows that integrate many data sources are most naturally expressed in high-level scripting languages. We describe the design of JutulDarcy, an open-source porous media simulator written in the Julia language. JutulDarcy is designed from the ground up with extensibility and computation of sensitivities in mind, starting from the assumption that high-performance automatic differentiation enables us to radically rethink the way simulators are designed. Highlights of this design include arbitrary execution graphs for the equation terms, in which dependence relationships can be re-configured on the fly, easy coupling of models, computation of sensitivities with respect to all model parameters through an adjoint method with a parameter declaration system, and an assembly process with performance that can exceed that of compiled simulators with hand-coded Jacobians. We validate the simulator on a number of standard test cases (immiscible, black-oil, CO2 and compositional flow) and demonstrate MPI parallel performance on multi-million cell models. In addition, we offer a frank assessment of the relative strengths and weaknesses of the Julia programming language for scientific computing.

Keywords Julia · Reservoir simulation · Compositional flow · Automatic differentiation · High performance computing

 https://doi.org/10.1007/s10596-025-10366-6

1 Introduction

In the ongoing energy transition, reservoir simulators are now increasingly tasked with simulating processes like CO_2 sequestration, geothermal energy, gas storage and thermal heat storage. The economic profitability of such projects is often marginal [1, 2] with less data available than in conventional geoenergy, and using improved simulators to assess outcomes, optimize operation strategies, and quantify uncertainty will play a large role during project planning and execution. Reservoir simulators are significant undertakings from a software development perspective, with long turnaround time both from initial development to first product, and when new features are to be added. Having sufficient flexibility to quickly model new processes and behaviors is essential to meet future simulation needs. One major research question in this direction is to what extent advances in compilers, programming languages and other generic software technology can improve the manner in which specialized reservoir simulation software is developed.

One example of such technology is that of automatic differentiation (AD), which is central to the current machine learning boom [3]. Although there are references describing the use of AD in a reservoir simulator as far back as 1999 [4], the most notable examples are the C++ AD-GPRS simulator [5], the MATLAB/Octave-based Matlab Reservoir Simulation Toolbox (MRST) [6, 7] and the C++ based OPM Flow simulator [8]. The use of operator based linearization in the DARTS simulator [9] can be considered as a form of AD with caching in a discretized parameter space. Although AD provides the developer with a large amount of flexibility, a straightforward application of AD can often lead to poor computational performance, and differences in the types of operations needed between reservoir simulation and machine learning makes the adaptation of existing high performance tensor AD libraries difficult. These challenges, in combination with extensive use of legacy codes written with manual implementation Jacobians in mind, mean that most reservoir simulation performed today is not done with the use of AD.

Although the terms are sometimes used interchangeably, AD is only one aspect of differentiable programming or differentiable simulation. Differentiable programming broadly refers to a methodology that allows computer programs to not only produce a numerical value for a given set of inputs, but to also produce the derivative with respect to these inputs. A myriad of different techniques, including variants of AD, are used as building blocks to achieve differentiability of a complex program. For example, the DiffTachi domain specific language (DSL) [10] is an early attempt at providing high performance, high productivity and differentiability for a wide range of simulation kernels that can be formulated in the DSL syntax.

A key aspect of differentiable simulation is not only that some form of AD is used in the program execution itself, but also that the outcome of the simulation can be differentiated with respect to the input parameters. The idea of being able to calculate sensitivities from a reservoir simulator is not new [11], and adjoint formulations have been implemented in MRST [6], AD-GPRS [12], the Chevron in-house simulator CHEARS [13], and DARTS [14]. There is also a fairly specific adjoint option in the commercial Eclipse 300 simulator [15] that is limited to a set of pre-programmed objective functions in the form of well rates and well bottom-hole pressures. Adjoint methods with respect to control variables (e.g., well rates) are generally less invasive to implement than those that also include petrophysical parameters, and this is reflected in the literature. Of the simulators mentioned here, only MRST and DARTS are publicly available as open source software. There is little reported operational use of reservoir simulators with adjoint capability and history matching workflows are in practice often based on methods that allow the use of the reservoir simulator as a "black box", for example in the ensemble Kalman filter approach [16].

In this work, we describe the motivations for designing a new differentiable reservoir simulator and discuss why Julia [17] was chosen as the programming language for a deep integration of AD in a reservoir simulator. There are other examples of reservoir simulation performed in Julia [18, 19], but our simulator `JutulDarcy.jl` is the first example of a general-purpose reservoir simulator capable to simulate cases of industrial complexity together with differentiable programming. Five examples are given to validate the simulator, demonstrate how variables and parameters can be added, sensitivities computed, and demonstrate computational efficiency.

2 Three motivations for differentiable simulation

The concept of differentiable simulation is taken to cover two different, but fundamentally related concepts: i) differentiation of the many smaller functions that described physical laws that constitute the building blocks of a simulator, which is typically the domain of automatic differentiation, and ii) the differentiation of the simulator itself as a function that maps inputs to outputs, which is often the domain of adjoint methods.

To motivate the examples in this paper, we will first discuss how the concepts can help in the development of increasingly complex simulator in broad terms. A reservoir simulator solves a system consisting of flow equations and accompanying closure relationships when advancing the solution. The basic flow equations are conservation laws on the form,

$$\frac{\partial m_c}{\partial t} + \nabla \cdot \vec{u}_c - q_c = 0, \tag{1}$$

where m_c is a conserved quantity, $\vec{u}_c$ the flux of that quantity, and q_c denotes source terms. The fluid systems in reservoir simulation generally consist of N_{ph} fluid phases made up of N_c components that may be either a single chemical species or a collection of such. To provide a background, we quickly review the most typical conservation laws that are all implemented in JutulDarcy.jl.

Conservation of phase masses In the case of $N_c = N_{ph}$ and each phase consisting of a single component, we can obtain the immiscible multiphase flow system with one equation per phase α,

$$\frac{\partial}{\partial t}(S_\alpha \rho_\alpha \phi) + \nabla \cdot (\rho_\alpha \vec{v}_\alpha) - q_\alpha = 0. \tag{2}$$

Here, ϕ is the rock porosity, ρ_α is the fluid mass density, $\vec{v}_\alpha$ the phase volumetric velocity, and S_α the phase saturation of phase α. For a single phase, (2) reduces to the scalar compressible pressure equation,

$$\frac{\partial}{\partial t}(\rho \phi) + \nabla \cdot (\rho \vec{v}) - q = 0. \tag{3}$$

Conservation of component mass Compositional systems can be broadly characterized by components being present in more than one phase. One example is the black-oil equations that can be seen as an extension of the immiscible case, where the assumption $N_c = N_{ph}$ remains, but the gas component can enter the oleic phase through the solution gas–oil ratio R_s and the oil component can be vaporized into the gaseous phase through the solution oil–gas ratio R_v. This system is most easily described by introducing the expansion/shrinkage factors b_α that relate the bulk volume of a pure component at surface conditions to the partial volume of the component in the same phase at reservoir conditions. These factors are used to account for the changes in fluid volume due to pressure and temperature variations between the reservoir and the surface, which allows a rewrite of the equations for the oil component,

$$\frac{\partial}{\partial t}\left[\phi \rho_o^s (b_o S_o + R_v b_g S_g)\right] + \rho_o^s \nabla \cdot (b_o \vec{v}_o + R_v b_g \vec{v}_g) - q_o = 0, \tag{4}$$

and analogously for the gas component,

$$\frac{\partial}{\partial t}\left[\phi \rho_g^s (b_g S_g + R_s b_o S_o)\right] + \rho_g^s \nabla \cdot (b_g \vec{v}_g + R_s b_g \vec{v}_o) - q_g = 0, \tag{5}$$

The densities at reservoir conditions for this model are written in terms of the shrinkage factors and the solution ratios for each phase:

$$\begin{aligned} \rho_g &= b_g(\rho_g^s + R_v \rho_o^s) \\ \rho_o &= b_o(\rho_o^s + R_s \rho_g^s) \end{aligned} \tag{6}$$

A more general two-phase compositional model is the liquid–vapor compositional model in which any number of components can be modeled, with phase properties and partitioning of components between the phases either described using an equation-of-state or as tabulated values:

$$\frac{\partial}{\partial t}\left[\phi(\rho_l X_i S_l + \rho_v Y_i S_v)\right] + \nabla \cdot (\rho_l X_i \vec{v}_l + \rho_v Y_i \vec{v}_v) - q_i = 0, \quad i \in \{1, ..., N_c\}. \tag{7}$$

Here, Y_i is the mass fraction of component i in the vapor phase and X_i the mass fraction of the same component in the liquid phase. Both the black-oil and the liquid–vapor compositional model can include a third immiscible phase, typically to model water.

Conservation of energy For systems that have significant thermal effects, a standard approach is to include the conservation of thermal energy. The equation, posed for a multiphase system for N_{ph} fluid phases and the solid rock phase with subscript r, reads,

$$\frac{\partial}{\partial t}\left(\rho_r U_r (1-\phi) + \sum_\alpha \rho_\alpha S_\alpha U_\alpha \phi\right) + \nabla \cdot \sum_\alpha (H_\alpha \rho_\alpha v_\alpha - S_\alpha \lambda_\alpha \nabla T) - \nabla \cdot (\lambda_r \nabla T) - q_e = 0, \tag{8}$$

This equation is independent of the exact form of the multiphase flow equations in use and can be combined with any of the aforementioned to simultaneously solve for heat and fluid flow. Thus, the number of components and phases matches that of the flow system it is combined with, with the additional terms internal energy density U, temperature T, phase enthalpy H_α, and thermal conductivity λ.

As primary closure relationships, we have assumed that the phase fluxes $\vec{v}_\alpha$ are given by the standard multiphase extension of Darcy's law, that phase saturations sum up to one, and that the phase pressures relate to some reference pressure via capillary pressures,

$$\begin{aligned} \vec{v}_\alpha &= -\mathbf{K}\frac{k_{r\alpha}}{\mu_\alpha}\left(\nabla p_\alpha - g\rho_\alpha \nabla z\right), \\ \sum_\alpha S_\alpha &= 1, \quad p_\alpha = p + p_{c\alpha}. \end{aligned} \tag{9}$$

Here, $k_{r\alpha}$ is the phase relative permeability, μ_α the phase viscosity, $\mathbf{K}$ the absolute permeability tensor, p_α the phase pressure and $g\nabla z$ the buoyancy forc

2.1 Implementation new models

The model equations presented thus far are not complete. Additional equations are necessary to determine how various physical parameters like b_α, $k_{r\alpha}$, $p_{c\alpha}$, etc., depend on the primary variables. These relationships can often be quite complex and will, in practice, be subject to more frequent changes than the basic conservation equations when moving from one particular physical case to another or when refining the model description.

We can see this if we take the volumetric multiphase Darcy phase flux $\vec{v}_\alpha$ as an example:

$$\vec{v}_\alpha = -\frac{k_{r\alpha}}{\mu_\alpha}\mathbf{K}(\nabla p_\alpha - g\nabla z) \tag{10}$$

If we were to drill further down into, for example, the definition of the viscosity μ_α, we would find many possible choices depending on the type of system or which phase we are considering, the type of mixture being simulated, and so on. This complexity is a fundamental part of reservoir simulation and makes the gap between research codes and industrial simulation large, as writing out the exact equations of interest can be difficult. Many of the relationships are also empirical and come in tabulated rather than closed form. We will return to this issue later on. For now, we simply note that as typical reservoir simulators rely on a large degree of implicitness in the temporal discretization and solution strategy, *derivatives* of each of individual term must also be implemented together with their discretization.

When developing a simulator, we first choose a suitable spatial and temporal discretization method, typically a fully implicit finite-volume method, which leads us to a nonlinear system of algebraic equations, which we can write on residual form as

$$\mathbf{r}_f(\mathbf{x}_f) = \mathbf{0} \tag{11}$$

for a given choice of primary variables $\mathbf{x}_f$. This system is generally solved using Newton–Raphson's method:

$$\mathbf{x}_f^{k+1} = \mathbf{x}_f^k - (J_f(\mathbf{x}^k))^{-1}\mathbf{r}_f(\mathbf{x}_f^k), \quad J_f = \frac{\partial \mathbf{r}_f}{\partial \mathbf{x}_f^k}. \tag{12}$$

To implement this, we must therefore not only hunt down suitable definitions from the literature for all terms and functional forms necessary to evaluate the residuals $\mathbf{r}_f$, but we must also differentiate them through with respect to the primary variables to obtain the Jacobian matrix J_f of the residuals, differentiated with respect to the primary variables. Doing this manually and analytically, is usually a laborious and error-prone process.

2.2 Extending existing models

Assume that we have succeeded in implementing a new model, but that we at a later stage need support for thermal effects in the form of (8). The heat and flow are tightly coupled, and thus a fully-coupled approach is natural. We therefore sketch out the new system that also solves the corresponding discretized residual thermal equation $\mathbf{r}_t$:

$$\mathbf{r}_{ft} = \begin{bmatrix}\mathbf{r}_f \\ \mathbf{r}_t\end{bmatrix}, \mathbf{x}_{ft} = \begin{bmatrix}\mathbf{x}_f \\ \mathbf{x}_t\end{bmatrix}, J_{ft} = \begin{bmatrix}\frac{\partial \mathbf{r}_f}{\partial \mathbf{x}_f^k} & \frac{\partial \mathbf{r}_f}{\partial \mathbf{x}_t^k} \\ \frac{\partial \mathbf{r}_t}{\partial \mathbf{x}_f^k} & \frac{\partial \mathbf{r}_t}{\partial \mathbf{x}_t^k}\end{bmatrix} = \begin{bmatrix}J_f & J_{ft} \\ J_{tf} & J_t\end{bmatrix} \tag{13}$$

Unfortunately for us, this simple extension represents more work than the initial flow solver: Not only do we have to implement the new thermal equations $\mathbf{r}_t$ and the Jacobian J_{tt} with respect to the thermal primary variables $\mathbf{x}_t$, we also need to derive and implement the cross-coupling blocks J_{tf} and J_{ft} in the Jacobian. This means that every single expression we previously have painstakingly differentiated with respect to the flow variables must also be differentiated with respect to temperature. If we were to add yet another equation type to this simulator, for example equations for geomechanics, even more cross-coupling blocks have to be implemented. The same principle applies when doing so-called hybrid modelling by for example replacing a constitutive relationship with a machine learning (ML) model, or coupling the discretization-based model to a differentiable ML model. The generalization of this thought experiment is that adding a single new equation to a model that already has N equations implemented means that we must implement both the discrete residual and the Jacobian of the new equation, as well as $2N$ cross-coupling Jacobian blocks. As we will see later, these problems can be overcome by

using automatic differentiation to automatically compute the necessary intermediate derivatives and assemble the Jacobian matrix or blocks.

2.3 The adjoint method for parameter sensitivities

In the two preceding subsections, we outlined few challenges of manually deriving and implementing the necessary derivatives in *forward* simulators that take a set of parameters, initial conditions, timesteps, and driving forces (well controls) as input and produces updated solution variables and quantities derived thereof as outputs. Often, parameters such as permeability or porosity are not fully known and must be calibrated to match observed fluid responses.

Model calibration is essentially an optimization problem in which the objective function is to minimize the mismatch between model predictions and observed data (or in some cases, data obtained from a more refined but more computationally expensive model). For optimization problems in general, having gradients available for smooth objective functions enables the use of algorithms having improved convergence, especially when many parameters are involved. A common method to obtain the necessary gradients is to use a so-called adjoint method.

We will briefly go over the adjoint method for a general optimization problem with respect to the parameters. Note that the derivation is conceptually similar if one is instead optimizing forces such as well controls. We define a set of n timesteps, assume that the initial state $\mathbf{x}_0$ is known, and let the primary variables $\mathbf{x}_i$ at each step i be defined by a converged set of residual equations with a time-dependent forcing term $\mathbf{f}_i$ for all steps $i \in \{1, \ldots, n\}$:

$$\mathbf{R}\left(\mathbf{x}_i, \mathbf{x}_{i-1}, \mathbf{p}, \mathbf{f}_i, \Delta t_i\right) = \mathbf{R}_i\left(\mathbf{x}_i(\mathbf{p}), \mathbf{x}_{i-1}(\mathbf{p}), \mathbf{p}, \mathbf{f}_i\right) = 0.$$

Assume that the objective function is a sum of distinct scalar objective functions for each step i, each depending on the primary variables of that step $\mathbf{x}_i$ and the static parameters $\mathbf{p}$ (but not on $\mathbf{f}_i$):

$$O = \sum_i^n O_i(\mathbf{x}_i(\mathbf{p}), \mathbf{p}) \tag{14}$$

An equivalent Lagrange function reformulation of this objective function for points in the physically feasible set $\mathbf{R}_i\left(\mathbf{x}_i(\mathbf{p}), \mathbf{x}_{i-1}(\mathbf{p}), \mathbf{p}, \mathbf{f}_i\right) = 0$ for all $i \in \{1, \ldots, n\}$ can be written as

$$J_\lambda = \sum_{i=1}^n \left[O_i(\mathbf{x}_i(\mathbf{p}), \mathbf{p}) + \lambda_i^T \mathbf{R}_i\left(\mathbf{x}_i(\mathbf{p}), \mathbf{x}_{i-1}(\mathbf{p}), \mathbf{p}, \mathbf{f}_i\right) \right].$$

If we differentiate this with respect to the parameters:

$$\frac{dJ_\lambda}{d\mathbf{p}} = \sum_{i=1}^n \left[\frac{\partial O_i}{\partial \mathbf{x}_i}\frac{d\mathbf{x}_i}{d\mathbf{p}} + \frac{\partial O_i}{\partial \mathbf{p}} + \mathbf{R}_i^T \frac{\partial \lambda_i}{\partial \mathbf{p}} + \lambda_i^T \left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i}\frac{d\mathbf{x}_i}{d\mathbf{p}} + \frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_{i-1}}\frac{d\mathbf{x}_{i-1}}{d\mathbf{p}} + \frac{\partial \mathbf{R}_i}{\partial \mathbf{p}} \right) \right] \tag{15}$$

If we now assume that the initial condition is independent of parameters, i.e., $\partial \mathbf{x}_0 / \partial \mathbf{p} = 0$, we can rearrange terms

$$\begin{aligned} \frac{dJ_\lambda}{d\mathbf{p}} &= \frac{\partial O_n}{\partial \mathbf{x}_n}\frac{d\mathbf{x}_n}{d\mathbf{p}} + \frac{\partial O_n}{\partial \mathbf{p}} + \mathbf{R}_n^T \frac{\partial \lambda_n}{\partial \mathbf{p}} + \lambda_n^T \left(\frac{\partial \mathbf{R}_n}{\partial \mathbf{x}_n}\frac{d\mathbf{x}_n}{d\mathbf{p}} + \frac{\partial \mathbf{R}_n}{\partial \mathbf{p}} \right) \\ &+ \sum_{i=1}^{n-1} \left[\frac{\partial O_i}{\partial \mathbf{x}_i}\frac{d\mathbf{x}_i}{d\mathbf{p}} + \frac{\partial O_i}{\partial \mathbf{p}} + \mathbf{R}_i^T \frac{\partial \lambda_i}{\partial \mathbf{p}} + \lambda_i^T \left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i}\frac{d\mathbf{x}_i}{d\mathbf{p}} + \frac{\partial \mathbf{R}_i}{\partial \mathbf{p}} \right) + \lambda_{i+1}^T \frac{\partial \mathbf{R}_{i+1}}{\partial \mathbf{x}_i}\frac{d\mathbf{x}_i}{d\mathbf{p}} \right] \end{aligned}$$

All terms involving $\mathbf{R}_i$ drop out since these equations are solved in the forward pass. We group terms by the Jacobian of the primary variables with respect to the parameters $\frac{d\mathbf{x}_i}{d\mathbf{p}}$:

$$\frac{dJ_\lambda}{d\mathbf{p}} = \left(\frac{\partial O_n}{\partial \mathbf{x}_n} + \lambda_n^T \frac{\partial \mathbf{R}_n}{\partial \mathbf{x}_n}\right)\frac{d\mathbf{x}_n}{d\mathbf{p}} + \frac{\partial O_n}{\partial \mathbf{p}} + \lambda_n^T \frac{\partial \mathbf{R}_n}{\partial \mathbf{p}}$$
$$+ \sum_{i=1}^{n-1}\left[\left(\frac{\partial O_i}{\partial \mathbf{x}_i} + \lambda_i \frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i} + \lambda_{i+1}^T \frac{\partial \mathbf{R}_{i+1}}{\partial \mathbf{x}_i}\right)\frac{d\mathbf{x}_i}{d\mathbf{p}} + \frac{\partial O_i}{\partial \mathbf{p}} + \lambda_i^T\left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{p}}\right)\right]$$

The Jacobians of primary variables with respect to the parameters are difficult to compute. To get rid of them, we simply choose Lagrange multipliers so that the adjoint equations are fulfilled, starting with the last timestep n:

$$\frac{\partial O_n}{\partial \mathbf{x}_n} + \lambda_n^T \frac{\partial \mathbf{R}_n}{\partial \mathbf{x}_n} = 0 \rightarrow \lambda_n = -\left(\frac{\partial \mathbf{R}_n}{\partial \mathbf{x}_n}^T\right)^{-1} \frac{\partial O_n}{\partial \mathbf{x}_n}^T$$

The adjoint equations for $i \in 1, \ldots, n-1$ can then be solved in reverse order, using the already obtained value of $\boldsymbol{\lambda}_n$:

$$\frac{\partial O_i}{\partial \mathbf{x}_i} + \lambda_i^T \frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i} + \lambda_{i+1}^T \frac{\partial \mathbf{R}_{i+1}}{\partial \mathbf{x}_i} = 0, \tag{16}$$

which can be solved recursively for λ_i from λ_{i+1} for the remaining steps:

$$\lambda_i = -\left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i}^T\right)^{-1}\left(\frac{\partial O_i}{\partial \mathbf{x}_i}^T + \frac{\partial \mathbf{R}_{i+1}}{\partial \mathbf{x}_i}^T \lambda_{i+1}\right) \tag{17}$$

Finally, we arrive at the expression for the gradient of the Lagrange function with respect to the parameters.[1] As we have assumed that the forward solutions ($\mathbf{x}$) used in this expression come from a sufficiently converged simulation, this expression is also the gradient of the objective function with respect to the parameters:

$$\frac{dJ_\lambda}{d\mathbf{p}} = \sum_{i=1}^{n}\left[\frac{\partial O_i}{\partial \mathbf{p}}^T + \frac{\partial \mathbf{R}_i}{\partial \mathbf{p}}^T \lambda_i\right]^T = \frac{dO}{d\mathbf{p}} \tag{18}$$

We remark that to accumulate the gradient using the back-propagation pass, we only need one *linear* solve per timestep from the original forward simulation, albeit with stricter numerical tolerances. Calculating the Lagrange function is a set of linear operations and could theoretically be much faster than nonlinear solves of the forward simulation. However,only a few nonlinear iterations are used per forward timestep in a typical reservoir simulation. Solving the adjoint linear systems and performing the additional linearizations of Jacobians to compute the gradient with the adjoint method is therefore of comparable cost to the forward solve.

The power of the adjoint method is apparent if we examine (17): Obtaining the gradient with respect to additional parameters by increasing the length of $\mathbf{p}$ only increases the size of some of the matrix-vector products and does not impact the linear solve in each step, which is typically the most expensive part for large problems of interest. However, there are a few challenges with applying adjoints in practical reservoir simulation: i) if the objective function is non-smooth, the adjoint method is not appropriate, and ii) the adjoint method requires additional functionality from the simulator that can be difficult and time consuming to implement. The main building blocks of the adjoint method are summarized in Table 1 together with an assessment of the relative difficulty of adding them

[1] We have assumed that the objective does not depend on the boundary conditions. The additional terms for this case are similar to those of the objective function with respect to the parameters.

 https://doi.org/10.1007/s10596-025-10366-6

Table 1 Overview of the different terms needed to implement the adjoint method for parameter sensitivities

Term	Usage	Notes
$\left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{x}_i}\right)^T$	Linear solve	Residual equations differentiated with respect to primary variables. The same type of linear system as in the forward simulation, except transposed. Requires a stricter solve than those in a typical forward loop, where a nonlinear residual is monitored and multiple linear solves are used per timestep.
$\left(\frac{\partial \mathbf{R}_{i+1}}{\partial \mathbf{x}_i}\right)^T$	Matrix-vector product	Residual equations differentiated with respect to previous time-step variables. Typically block-diagonal for implicit systems, as the equations only depend on the previous step through accumulation terms.
$\left(\frac{\partial \mathbf{R}_i}{\partial \mathbf{p}}\right)^T$	Matrix-vector product	Residual equations differentiated with respect to model parameters. These derivatives are not normally used in a standard forward simulation and are very time-consuming to implement without automatic differentiation since most reservoir simulators have more parameters than primary variables.
$\left(\frac{\partial O_i}{\partial \mathbf{x}}\right)^T$	Element-wise addition	Contribution at step i to the objective function differentiated with respect to the primary variables. The sparsity depends on the type of objective function, ranging from very sparse if the objective uses only well responses, to a dense vector for quantities like field-averaged pressure.
$\left(\frac{\partial O_i}{\partial \mathbf{p}}\right)^T$	Element-wise addition	Contribution at step i to objective function differentiated with respect to parameters. See above.

Rows that are colored **green** correspond to terms that are easily adapted from the existing routines of a regular simulation, **orange** are terms that contain substantial effort to implement, while **red** terms are gradients of the user-provided objective function and need to be differentiated with respect to the internals of the simulator

to an existing forward simulation code. For an overview of the use of the adjoint method in reservoir simulation, see the review by [20].

3 Implementation of differentiable simulator

It is at this stage prudent to clarify what is meant by a fully differentiable simulator, as there is some nuance in exactly what this fully encompasses. We define a reservoir simulator to be fully differentiable if it is able to give gradients of user-provided objective functions, i.e., some quantity derived from the output variables, with respect to parameters and forces (well controls, boundary conditions, etc.). The objective function could also be the output variables themselves, for example the maximum grid-block pressure in caprock cells that are at risk of fracturing. In addition, inclusion of user-provided functional forms, for example, a new density relationship as a function of pressure, should not require manual derivation and implementation of derivatives to work with both a forward simulation and producing gradient. We do not include differentiability with respect to hyperparameters such as linear solver tolerances, compiler optimization level, and choice of spatial and temporal discretization.

The three motivations we have discussed in the preceding section cover different aspects of simulator development: implementing new models, extending existing models, and computing adjoints are a limited subset of challenges when implementing reservoir simulators, but they share a common thread in that the difficulty of executing each of them is tightly connected to how derivatives are computed. Implementing derivatives and Jacobians manually is time consuming and error prone, often more so than the residual equations themselves. As we saw in the derivation of the adjoint method, an approach based on manual implementation of derivatives limits the utility of calculated gradients as it is difficult for users to provide custom objective functions without access to internals of the simulator.

3.1 Automatic differentiation for differentiable simulation

AD refers to a wide class of techniques for obtaining derivatives of numeric output from computer programs with respect to the input arguments. The core idea of AD was first published in the 1960s with [21] describing what we would now refer to as forward mode AD, which is in widespread use for many scientific applications, with training of machine-learning models being the perhaps most well-known example [3].

Most readers who have encountered a tutorial on automatic differentiation have at least passing familiarity with the scalar-to-scalar case: Say that we have a scalar to scalar function for density that, for a given numerical value of pressure, produces the corresponding density $\rho(p)$. Here, the use of some form of AD would, without manually coding an additional function, produce the derivative $\frac{\partial \rho}{\partial p}$ for the specific value of p passed in. This is in contrast to numerical differentiation, which would *approximate* the derivative at the point using a finite difference scheme, or to symbolic differentiation that would get an analytic expression for the derivative of the function itself ρ. The exact technique used to obtain this derivative is not essential for our discussion, but we note that there are variety of different methods, ranging from forward/dual number AD, reverse mode/tape-based AD, and source-to-source transformation; see [22] for more details.

AD is the main building block of a differentiable simulator. One approach is to take the discretized residual version of (1) as a "black-box", vector-valued function of the primary variables $\mathbb{R}^n \rightarrow \mathbb{R}^n$ and apply any kind of generic automatic differentiation. The typical discretizations in reservoir simulation are, however, generally sparse in that the equation of a specific cell or face only depends on a few nearby neighbors. For example, a two-point flux approximation on a structured mesh will give a seven point stencil. In practice, the stencil varies a lot from cell to cell when corner-point or unstructured meshes are used. We can make a very simplified estimation: Assuming that calculating one derivative with AD has a cost of the same order of magnitude as the computation of one value of the residual, the total cost will be of the magnitude n^2. A direct application of AD to this vector valued function will therefore not be efficient, and exploiting this irregular sparsity is the key to efficient use of automatic differentiation. One high-level approach to this end is Jacobian coloring to reduce the number of evaluations in the presence of sparse, but overlapping stencils [23].

3.2 Goals of a differentiable simulator

Even as there is a fair bit of use of both AD and adjoint methods in research, it is far from being a standard way of writing simulators. One contributing factor is that most reservoir simulators in use are built upon legacy codes that have been gradually extended with a large number of features: Commercial-grade reservoir simulators are by nature highly complex and there is a significant barrier to entry for writing a new simulator, or to update the core parts of an existing code. We outline three key qualities that should be present in a fully differentiable simulator:

1. **Extensibility:** The simulator must easily admit new functional relationships, and adding a new parameter or variable should ideally be just a few lines of code. Most codes are either closed source, in which extension is impossible, or sufficiently complicated that adding new functional relationships requires a substantial effort. We see open source as a necessary prerequisite for extensibility, but open source is not sufficient by itself.
2. **Accuracy:** Reservoir models are complex to set up and simulate correctly. The usefulness of a fully differentiable solver is limited if it does not produce the same results as commercial-grade simulators for cases of industrial relevance. Implementing a differentiable simulator for, e.g., simple two-phase incompressible and immiscible problems with analytical input functions is possible as a student project, but will be limited in how complex models it can run.
3. **Efficiency:** A differentiable simulator should be comparable to a commercial-grade simulator in terms of computational performance. By comparable, we mean that runtimes should be of the same order as the tools in

use today and not orders of magnitude slower. The biggest utility of a differentiable simulator is in workflows that require a great many simulations, and the impact is limited if this is not possible on standard models.

For the reasons mentioned, we believe that the impact of a fully differentiable simulator will be severely limited if either of these three goals is not met. In this respect, it is more important to be reasonably efficient and meet all goals, than to have the world's fastest differentiable Cartesian mesh oil–water simulator.

3.3 Jutul and the Julia programming language

What is now called Jutul (`Jutul.jl`) started in 2021 as a part of a series of experiments in development of high-performance AD functionality. The MATLAB-based automatic differentiation library from the open-source MRST software was extended from porous media simulation to battery simulation in the form of the BattMo [24][2] framework. Simultaneously, the AD library originally designed for reservoir simulation was used for projects within carbon capture and other non-reservoir models. A common feature of these applications was that the mesh sizes were generally smaller than in typical reservoir applications, but with additional complexity in the form of varying equations in different parts of the domain. Evaluation of a lot of small, hard-to-vectorize functions constitute a worst-case scenario for scripting languages like MATLAB that rely on vectorized calls for efficiency, and the runtimes for these models were often unacceptably high when doing optimization and parameter matching.

MRST had been public for more than a decade at this point [25], and even as the code had been under active development, including the more recent integration of automatic differentiation [6, 26], it was still using MATLAB as the foundation. Certain parts, including parts of the AD library and linear solvers, had been developed are more high-performing backends written as extensions in C++ [26]. This improved performance significantly but also reduced the user experience as compiler setup on different platforms can be brittle and complicated. It is also more difficult to gain an understanding of the code base when multiple languages are introduced. We remark that the reverse situation is also seen, where a high-performance code written in Fortran or C++ eventually gets Python or MATLAB bindings bolted on for more flexible usage. Both of these situations are symptoms of what the authors of the Julia programming language refers to a the *two-language problem* [17]. There has always been a tension between the programmer and compiler in scientific computing, where compiled languages like Fortran and C/C++ are fast to execute and slow to write, and high-level languages like MATLAB or Python are quick to develop programs in, but often slower to execute. This gap is far from settled, as between 2021 and 2024 there has been at least one high-profile language launch promising to provide the "usability of Python with the performance of C" in the form of Mojo [27].

The software landscape had evolved significantly during the decade from 2010 to 2020: Python had for many applications displaced MATLAB as the the de facto scientific scripting language, and also was the primary interface to most of the high-level machine learning frameworks. Although previously infamous for poor performance, Python could be accelerated by use of JAX or Numba. In addition, entirely new languages like Swift, Rust, and Julia had risen into maturity. The goal was to evaluate options for a language with high-level scripting capabilities similar to MATLAB or bare Python, but with better performance for conceptual models with representative features from practical problems.

After a series of explorations and surveys, Julia was found to be the most promising candidate in 2021. Early tests of porting parts of MRST were performed as early as 2017 when version 0.5 of the language was found lacking, with a later test of version 1.0 for reservoir simulation in a master thesis [18] providing a more positive outlook. The main reasons for going further with Julia was: i) a well-developed, if somewhat fractured, AD ecosystem, ii) excellent performance for complex codes similar to reservoir simulation kernels, and iii) emphasis on linear algebra and preconditioners. In addition, although the outlook for a relatively new programming language was uncertain, the potential benefits of writing both exploratory scripting code and high-performance inner loops in the

[2] Freely available on GitHub: github.com/BattMoTeam/BattMo

same language are significant. The result was `Jutul.jl`[3], a framework for writing Newton solvers for multiphysics simulators based on automatic differentiation, and the packages `BattMo.jl` and `JutulDarcy.jl` that make use of the fundamental infrastructure in `Jutul.jl` in battery and reservoir simulation, respectively.

3.4 Differentiable simulation with `Jutul.jl`

A full description of the inner workings of `Jutul.jl` is outside the scope of this paper, but we will briefly go over the design as background to the later results produced using `JutulDarcy.jl`. All models in the Jutul framework are defined by variables, entities, and equations:

1. A model contains a runtime-defined *variable graph* made up of a set of three types of variables: primary variables are solution variables, parameters are static values, while secondary variables can be computed from a combination of primary variables, parameters, and other secondary variables. Each type of variable is associated with a spatial *entity* (e.g., a face, cell, or well perforation) and only depends on quantities at that same entity. Secondary variables are stored so that values can be used in multiple equations without recalculation.
2. Each model has any number of equations, also associated with entities that can use variables from *any* entity to form a discrete residual equations that can be checked for convergence. For example, a reservoir model will typically have a mass-conservation equation for each component (or phase) that for a given cell depends on variables from the neighboring cells and parameters on the faces of that cell.
3. Multiple models can be coupled together with any number of so-called cross-terms that depend on the variables of both models in the same way as equations depend on the variables of one model.

Once a model is set up together with an initial state of primary variables and parameter values, a simulator object is instantiated. The dependency graph of all variables is sorted, ensuring that all secondary variables can be updated in sequence by starting from the primary variables and parameters without cyclical dependencies.

Variables are stored as structs of arrays, with one vector per variable (or matrix, if multiple values exist per entity, e.g., for phase densities). Storage is allocated for derivatives of variables and equations. In the process, all equations are evaluated with a special sparse tracer type that determines impact of an equation's i-th entry with respect to all other entities. This is done using a re-purposing of the algorithm from [28] to trace the impact of entities instead of values. The specific implementation is from the `SparsityTracing.jl` Julia package [29].

The sparsity information is used to allocate storage for a sparse automatic differentiation pass using the dual number AD type from `ForwardDiff.jl` [30]. The AD type used is a scalar value and a dense vector with the equal number of partial derivatives to the number of degrees of freedom in a single entity. A wrapper type on the set of variables can be used to get derivatives of expressions with respect to a single entity, for example the component fluxes at a face with respect to pressure and compositions of a specific cell.

A (block) linear system with the same sparsity pattern is also allocated, and the positions of each nonzero derivative into the linear system vector of nonzero entries is stored. This last alignment stage allows for fast updating without searching in the matrix. This storage is also specialized for general conservation finite-volume laws, where the stencil of each discrete cell-to-cell flux determines the sparsity pattern, and the number of AD evaluations can be significantly reduced by exploiting the symmetry of the fluxes, where flow entering one cell is leaving another. This specialization is taken a step further the case where the flux is a two-point scheme where the flux only depends on exactly two cells in a storage format inspired by the assembly of OPM Flow [8].

The simulation uses a Newton loop with a variety of options for limiting changes in variables, as typical for reservoir simulation. Once a simulation is started, the simulator first updates all secondary variables based on the current values of the primary variables, then updates all residual equations, coupling terms, and their corresponding derivatives. Forces are applied to the equations. If any equations are determined to not yet be converged, a linear

[3] See github.com/sintefmath/Jutul.jl and jutul.org

solve is performed, and primary variables are updated before the process repeats. During a timestepping loop, all calls relating to variables, equations, coupling terms, and linear systems are executed using mutation instead of assignment. The advantage of the pre-allocated buffers and sparsity pattern is highly efficient updates, with memory being updated sequentially with the same function call, and the same types. The disadvantage is that the design assumes that the sparsity pattern is fixed throughout the simulation. If this was to change, for example in the case of adaptive mesh refinement, the sparsity would have to be re-detected and a new linear system set up.

3.5 Julia development experience

Julia was first publicly released in 2012. After undergoing substantial syntax and feature changes during the first years of development, the 1.0 version was released in 2018 with the promise of backwards compatibility for both syntax and standard library API. The Julia compiler uses LLVM and is, under the right conditions, able to produce fast machine code on the same level as traditional statically typed compiled languages like C, C++, or Fortran. Julia 1.5 was the latest version when `Jutul.jl` started development in 2021.

Julia uses just-in-time compilation, where compilation can occur during the execution of a program if a new function is encountered, or an already known function is called with alternative input types, for example when switching from floating point types to AD types. Functions where the compiler is unable to narrow down the types of the output – so-called type unstable calls – will still execute, but performance will degrade to that of a slow interpreted scripting language. As a result, performance of Julia code depends on the compiler being able to figure out down what the output type of a function will be when called with a specific combination of input types.

For general scripting work, this performance penalty is not noticeable, but assessing type stability is essential for performance critical functions, like the innermost calls of equation assembly or linear solution. A significant advantage of this programming model is that prototype code can be written without regard for performance, and subsequently optimized if performance becomes an issue. The disadvantage is that there can be significant delays or stops in execution when new functions are compiled behind the scenes. This is most readily apparent in the "time to first plot"-issue, meaning that there can be a significant delay between starting a Julia session with many loaded packages and first program execution, although this issue is less of an issue in later Julia versions where packages cache compiled code between sessions.

Although getting the best possible performance out of Julia code requires a fair understanding of technical programming concepts like mutation, sequential memory access, and type inference, much of the code executing during simulation in `JutulDarcy.jl` could easily be passed off as Python or MATLAB code with minor modifications to syntax. A particular quirk of reservoir simulation is that there are a very large number of constitutive relationships that must be evaluated efficiently during simulation. Most of these functions take numerical inputs (e.g., pressure and temperature) and return a value of the same type (e.g., density), and such functions are generally efficient without the use of advanced techniques.

Picking a programming language is always a balance between different considerations. Next, we will go over our experienced advantages and disadvantages in using Julia for scientific software development that may be useful for anyone who is curious about Julia.

Advantages

- Code is fast to execute, as in a compiled language, even with custom types and functions, and there is support for common HPC hardware execution through `CUDA.jl`, `AMDGPU.jl` and `MPI.jl`.
- The Julia package manager is integrated into the language and makes it easy to install software and manage dependencies.

- The package ecosystem has a large number of useful packages. Many existing libraries written in other programming languages have platform-independent wrappers, making it easy to get access, e.g., to proven preconditioners or standard optimization solvers.
- Julia code is often *composable*, so that it is easy to combine functions from different developers. For example, the linear solvers in `JutulDarcy.jl` for MPI uses a Krylov implementation together with a distributed array implementation, where each package was implemented independently by different authors.
- Visualization and plotting packages are excellent, and a script can go from high-performance computing to visualization without any friction.
- Writing a Jupyter notebook or a cell-by-cell script in VSCode makes it easy to interactively work with data when writing a paper or performing analysis.

Disadvantages

- The package system has a large number of packages registered, and many packages are immature or under-documented. It can be hard to navigate the ecosystem when looking for a specific feature.
- First-time loading of new packages can still take a long time, as they are compiled locally on the users machine.
- The default Julia debugger is quite slow and brittle, and while there are configurable options that make the experience better, this area is lacking compared to other languages.
- The fact that the efficiency of code execution depends on the compiler's type inference means that it can be challenging to determine exactly why a piece of code is slow or allocates memory that must be garbage collected. As the program will run even if code is not type stable, it is possible to introduce slowdown in an existing code by inadvertently adding type instability during refactoring or feature updates if performance is not monitored closely.

Neutral points

- The compilation model takes some time to get used to. Unless the program is completely type stable and carefully manages memory, the size of compiled binaries will be several hundred megabytes and require a representative workflow during compilation. However, it is surprisingly simple to produce an executable from a complex script and distribute it without much knowledge about how to build executables.
- Julia is not a object-oriented language, but rather uses abstract types and multiple dispatch as the main mechanism for extending types and functionality. Although multiple dispatch is a powerful tool for program design, there is a significant learning curve when transitioning from object-oriented codes.
- Julia is a specialized and relatively new language. This is both an advantage in that the focus is largely on scientific computing, but can also be a disadvantage relative to, for example Python, for which significantly more resources are devoted to the user experience.

All over, we believe the advantages outweigh the disadvantages for differentiable reservoir simulation. The expressiveness of Julia code allows for compact, extensible code with high performance: `Jutul.jl` and `JutulDarcy.jl` are both less than 20,000 lines of code (LOC), with the black-oil and compositional specializations being less than 1,500 lines each. The entire well and facility model fits in 2,215 LOC, and the thermal implementation is less than 400 LOC.

 https://doi.org/10.1007/s10596-025-10366-6

4 JutulDarcy

`JutulDarcy.jl` is written entirely in Julia and is therefore supported on all major operating systems where Julia can be installed. The code is released under the MIT license and is registered in the official Julia package registry. This means that the package can be installed by name through the builtin Julia package manager that automatically handles installation of the package itself, and of all dependencies for the users current Julia version and operating system:

```
using Pkg; Pkg.add("JutulDarcy")
```

4.1 Development model

The code is developed using git and development is hosted publicly on GitHub. Jutul and JutulDarcy are developed as separate packages[4], with JutulDarcy using Jutul as a dependency. The version of Jutul used when you install JutulDarcy will be handled by the Julia version manager.

4.1.1 Testing and CI

The package uses extensive tests to ensure stability across releases, with test suites that cover all supported physical systems and features. The tests can be run locally by the user, but are also run automatically on the GitHub Actions continious integration (CI) on every pull request and push to the main branch. The tests are a mixture of unit tests, integration tests and regression tests.

4.1.2 Documentation and literate programming

JutulDarcy is documented using the Julia `Documenter.jl` package. Documenter uses a mixture of Markdown and code that execute using the documentation build process, similar to e.g. Sphinx for Python packages. The documentation is hosted on GitHub Pages and is built automatically as a part of the test system, allowing pull requests to be reviewed with a preview of the changes to the documentation itself.

One hurdle of hand-written documentation is that it can quickly become outdated. A similar issue exists for usage examples and notebooks that are not part of the test suite. JutulDarcy uses literate programming [31] to mitigate this issue, where code is interleaved with explanatory text and mathematical equations. This means that large parts of the functionality is demonstrated and tested in the documentation itself. In practice, this is done by writing scripts with cells deliniated by Markdown comments that explain each code section, with tests being interleaved in the code blocks. These examples are standard Julia scripts where changes are easily tracked by version control, but the code can also be translated by the `Literate.jl` package into Jupyter notebooks or the preferred Documenter format where comments are translated into headers, text blocks, LaTeX equations, and code blocks. At the time of writing, JutulDarcy contains such 25 examples, including the examples contained in this paper, with the exception of performance tests that take too much time to run on CI and may make use of non-open models.

4.2 Supported physical systems

`JutulDarcy.jl` supports the types of physical systems described in the motivation section. In addition, K-value compositional models are supported, with a tailored CO_2-brine, thermal K-value model included that matches the

[4] Repositories: github.com/sintefmath/Jutul.jl github.com/sintefmath/JutulDarcy.jl

description of the 11th SPE Comparative Solutions Project on CO_2 storage described in [32]. Boundary conditions, source terms, and both regular and multisegment wells are supported. Wells are described as full numerical models in their own right with parameters and governing equations, and can be simulated independently of the reservoir. The facility model that operates the wells can have time-varying controls and multiple operational limits, with dynamic switching of active controls as a response to violated limits during simulation. The number of perforations and their connection strength to the reservoir can vary throughout the simulation, and options for well downtime is supported. The solver includes a number of features that are often omitted from research codes like relative permeability endpoint scaling, heterogenous capillary pressures, spatially varying fluid and PVT regions, and initialization through equilibriation. As an example of how the code looks in practice, we can examine a short excerpt of the code that computes the total component mass for a black oil system. This is the accumulation term of from (4) and (5) for a two-phase system:

```
function update_blackoil_mass!(M, pv, b, S, Rs, Rv, rhoS, sys)
    Φ = pv[cell]                              # pore-volume
    l, v = phase_indices(sys)                 # 1, 2 or 2, 1
    bO, bG = b[l, cell], b[v, cell]           # get b-factors
    sO, sG = S[l, cell], S[v, cell]           # get saturations
    M[l] = Φ*rhoS[l]*(bO*sO + bG*sG*Rv[cell]) # oil component mass
    M[v] = Φ*rhoS[v]*(bG*sG + bO*sO*Rs[cell]) # gas component mass
    return M
end
```

Here, `b`, `S`, are $N_{nph} \times N_{cell}$ matrices that hold b and saturations for each phase, and `Φ`, `Rs`, `Rv` are vectors with N_{cell} entries that hold the pore volumes, R_s and R_v, respectively. The exclamation mark in the function signature is a Julia convention that indicates that the function mutates and returns the first argument. The code maps directly to the mathematical expressions, is array-based, lacks verbose annotations about types or template arguments and is still highly efficient. We note that for the purposes of the paper, this code has been simplified to remove conditional branches for the absence of R_s/R_v and the possible presence of a aqueous phase.

4.3 Input and output formats

Reservoir simulation models can be written entirely as Julia scripts, exported from MRST, or loaded from Eclipse-type input files. In the latter case, a number of keywords and features are supported, with warnings given when non-supported features are encountered. A corner-point parser with support for processing of faults, pinched layers, and non-neighbor connections is included. Input files from Eclipse-type or MRST input are converted to the same data structures as those used in a standalone script, making it easy to, for example, add new wells or alter properties as a part of a custom workflow. A workflow can be developed in a rapid iteration loop for a simple conceptual setup that takes seconds to run, before substituting the model for a complex input case that takes hours to run when the workflow is finished. Output files are written in JLD2, a HDF5 dialect.

4.4 Linear and nonlinear solvers

The linear solver is a pure Julia implementation of a constrained pressure residual preconditioner (CPR) with block-ILU(0) as the second stage preconditioner. The CPR has a specialized option tailored for solving adjoint systems similar to the preconditioner described in [13]. The linear solver calls the `Krylov.jl` module [33] for Krylov subspace acceleration. Wells and facilities are handled by a Schur complement reduction without fill-in. Standard Julia matrix and vector formats are used, making it easy to test new preconditioners that are implemented in Julia. Extensions make it possible to run linear solves in parallel using MPI or on GPUs. The nonlinear systems can be solved by either Newton's method or nonlinear domain decomposition (NLDD) [34].

4.5 Extensions, add-ons and advanced usage

Additional packages can be installed to automatically extend the code with additional functionality. We give a brief overview in Table 2 of a few examples of such packages. In particular, MPI parallelism and GPU acceleration is handled by loading additional packages that provide alternative implementations of Arrays that are distributed between processes or reside in device memory. It is also possible to compile a standalone executable with a command line interface, which can be useful when running many simulation models or when using MPI.[5] The code can also be called from Python via a wrapper package that produces standard NumPy arrays.[6] An optimization interface is included for coupling the simulator to different optimization packages with handling of limits, and efficient reuse of in-memory simulator data structures during the optimization process.

5 Examples

In the following sections, we will present a series of examples to highlight the software's functionalities and capabilities. Figure 10 contains plots of meshes, wells and porosity for a number of public models that will be used in the examples. All examples are simulated on a consumer grade PC with a AMD Ryzen 9 7950X 16-Core Processor with 128 GB of memory. The GPU is a NVIDIA RTX 4080 with 16 GB of memory. Unless otherwise noted, timings are reported in serial running on CPU. This section is supplemented by a large number of figures and tables that show performance and accuracy comparisons, found in the Appendix. The examples in this paper, with the exception of those that make use of non-public models, are available as a part of the released version of JutulDarcy, and can be written to disk by running the following code in a Julia REPL:

```
using JutulDarcy
generate_jutuldarcy_examples()
```

5.1 Introductory example

We will cover several different types of examples in this paper, and we will occasionally show excerpts of code to demonstrate how particular results were achieved. In the interest of making these later excerpts easier to digest, we will first set up, simulate, and visualize a small problem with two wells in a quarter-five-spot configuration with all code documented. We start by installing the prerequisite packages and an optional plotting package:

```
using Pkg
Pkg.add(["Jutul", "JutulDarcy", "GLMakie"])
```

The installation of packages is typically done once per environment, and the first install will take some time as all dependencies are installed and precompiled. Once the packages are installed, we can begin the example itself by accessing a few units (converted from field to SI units) and setting up an immiscible two-phase system, with the first being a liquid and the second being a vapor phase. Reference densities of 100.0 and 1000.0 kg/m^3 are set and is used by the default evaluation of density and to define surface volumetric rates through wells:

[5] github.com/sintefmath/JutulDarcyApps.jl
[6] github.com/sintefmath/PyJutulDarcy

```
using JutulDarcy, Jutul, GLMakie
d, bar, kg, m = si_units(:darcy, :bar, :kilogram, :meter)
dy, s = si_units(:day, :second)
rhoLS, rhoGS = 1000.0*kg/m^3, 100.0*kg/m^3
sys = ImmiscibleSystem(
    (LiquidPhase(), VaporPhase()),
    reference_densities = (rhoLS, rhoGS)
)
```

We next define the reservoir itself by setting up a Cartesian mesh with $100 \times 100 \times 1$ cells that discretizes a $2000 \times 2000 \times 100$ meter box. We specify a scalar permeability of 0.1 darcy and a porosity of 0.2 and set up our reservoir domain that contains all the static petrophysical parameters needed for simulation. These scalar properties are automatically expanded to all cells of the domain, but we could also have provided vectors as inputs, or in the case of permeability, a matrix.

```
nx, ny, nz = 100, 100, 1
g = CartesianMesh((nx, ny, nz), (2000.0, 2000.0, 100.0))
reservoir = reservoir_domain(g, permeability = 0.1d, porosity = 0.2)
```

We place a pair of wells in each corner of the domain. We call two different setup functions for wells, one for vertical wells for the producer and a general one for the injector that takes in a set of cell indices; as the mesh is logically Cartesian, we can directly pass an IJK triplet to select a cell. Although JutulDarcy supports complex simulation schedules with multiple limits per well and varying controls per timestep, the setup here is intentionally simple, with a fixed injection rate and production at a constant pressure.

```
P = setup_vertical_well(reservoir, 1, 1, name = :P)
P_ctrl = ProducerControl(BottomHolePressureTarget(50bar))
I = setup_well(reservoir, (nx, ny, 1), name = :I)
rate = TotalRateTarget(1.5m^3/s)
I_ctrl = InjectorControl(rate, [0.0, 1.0], density = rhoGS)
dt = fill(30.0dy, 60) # 5 years, 12x5 months
```

Now that we have a reservoir, wells, and the physical system ready, we can set up the model and parameters. We also set up an initial state consisting of a liquid-filled reservoir at constant pressure. At this stage, we could also customize our model by setting different evaluators for secondary variables like density, viscosity, or relative permeability, but we can also leave them defaulted to get a weakly compressible model with linear relative permeability curves and constant viscosity.

```
m, p = setup_reservoir_model(reservoir, sys, wells = [I, P])
s0 = setup_reservoir_state(m, Pressure = 150bar, Saturations = [1.0, 0.0])
```

With everything set up, we instantiate a set of forces based on our well controls and simulate the model. The resulting plot interface is shown in Fig. 1. This code is free of type declarations, and here Julia is essentially used as a scripting language. At only 22 lines of code for a complete reservoir simulation, the syntax should be quite familiar to anyone who has spent time writing Python or MATLAB code.

```
f = setup_reservoir_forces(m, control = Dict(:I => I_ctrl, :P => P_ctrl))
ws, states = simulate_reservoir(s0, m, dt, parameters = p, forces = f)
plot_reservoir(m, states, step = length(states), key = :Saturations)
```

5.2 Running .DATA input files

The .DATA file format, first used by Eclipse, is a de facto standard for reservoir simulation models and the format and variants thereof can be read and written by different tools. We will demonstrate how a few widely used test

 https://doi.org/10.1007/s10596-025-10366-6

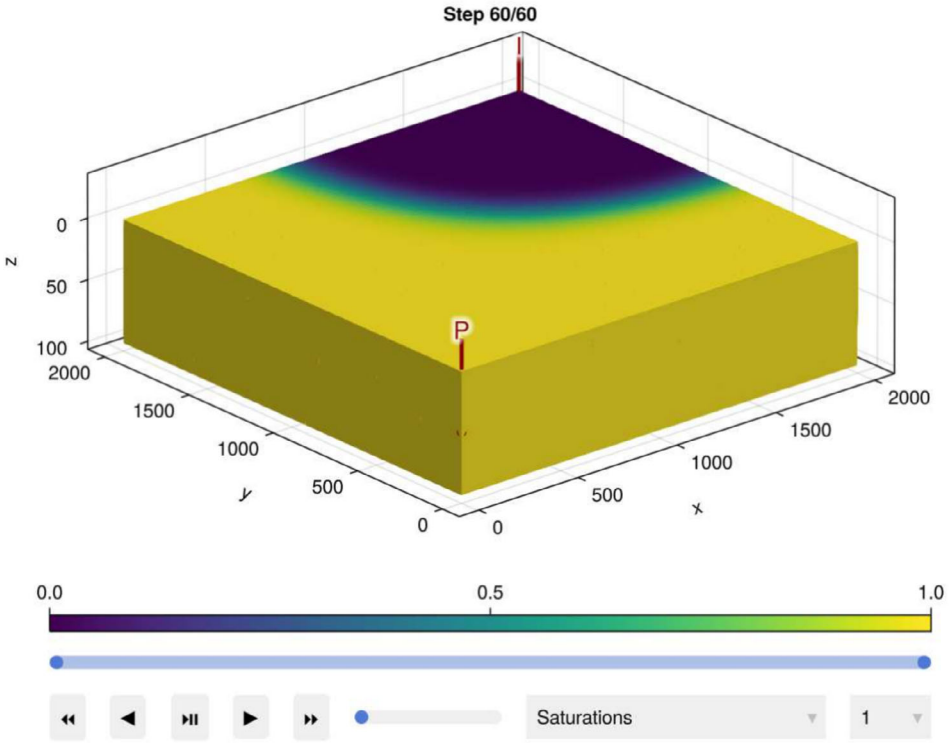


Fig. 1 Example of a quarter-five-spot saturation front set up in the introductory example. The graphical user interface uses `GLMakie.jl` for 3D visualization and interactive controls

cases can be read and simulated by `JutulDarcy.jl`, before we compare the simulation results to those from two operational reservoir simulators.

5.2.1 The SPE 9 model

The first model is the SPE 9 case originally described by [35] with the specific input file being taken from the OPM test data repository.[7] The model has a small, 9,000-cell mesh with 26 producers and a single injector. The fluid is a three-phase black-oil system with dissolution of gas into the oleic phase. The reservoir is initially filled with undersaturated oil, and gas comes out of solution as a response to the pressure drop during production. The reservoir is plotted in Fig. 10a and the well responses are compared against Eclipse results in Fig. 11. We observe an excellent match, with agreement on the level of individual wells, including when oil production declines as well as during dynamic well-control switches.

5.2.2 The OLYMPUS model

The second model is the first realization of the model ensemble from the OLYMPUS optimization challenge [36]. The reservoir is represented on a $118 \times 181 \times 16$ corner-point grid with 192,749 active cells. The model is faulted and uses transmissibility multipliers to alter the flow across these faults. The flow model describes a compressible, two-phase, oil-water system with three different rock types and relative permeability curves. The wells operate according to a typical waterflood, where seven injectors have constant water rates and eleven producers operate at fixed bottom-hole pressure.

[7] See `github.com/OPM/opm-tests`. Licensed under the Open Database License.

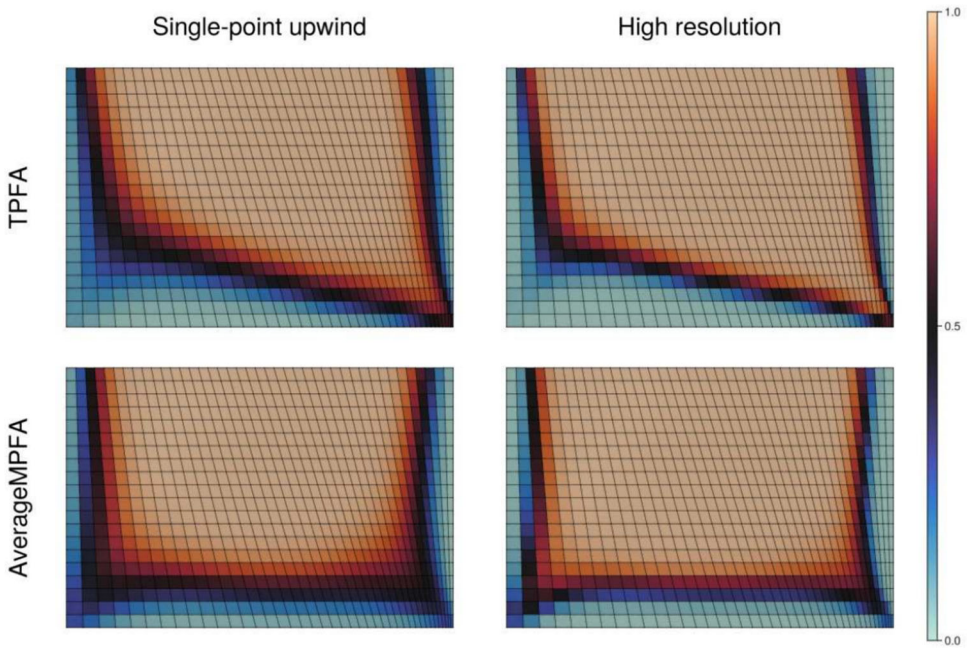


Fig. 2 Four different combinations of upwinding and gradient operators for an intentionally skewed mesh. The displacement is sensitive to numerical diffusion and benefits from a high-resolution WENO scheme

The code is the same for both models, and the simulator is used with default options for timesteps, convergence criteria, and linear solvers. The only modification from the previous example is that `HYPRE.jl` is loaded for improved performance on the OLYMPUS model.

```
# Load modules
using JutulDarcy, HYPRE
pth = "SPE9_CP.DATA" # Or "OLYMPUS_1.DATA"
case = setup_case_from_data_file(pth) # Setup model and schedule
ws, states = simulate_reservoir(case) # Perform reservoir simulation
using GLMakie # Load plotting
plot_well_results(ws)
```

5.3 Alternative discretizations

We have discussed that AD simplifies the extension of codes to new physics, but the same applies equally well to the introduction of new discretizations. If a code is based on AD, implementing a new expression for a flux will not require additional derivatives to be implemented manually, and composing different discretizations can be done without additional effort on part of the programmer. JutulDarcy makes use of the industry-standard discretization in the form of two-point flux approximation (TPFA) for the pressure and a single-point upwind (SPU) scheme for upwinding. This combination of schemes is remarkably robust, and even in the presence of known shortcomings for certain grids and displacement types, they are widely used both in academia and industry. JutulDarcy also includes a few alternative schemes that can be enabled, and are generally more accurate, but at the cost of being less robust and more expensive to linearize. This includes a family of linear and nonlinear schemes for the pressure approximation that cover the so-called nonlinear TPFA (NTPFA), average MPFA (AvgMPFA) and nonlinear MPFA [37–39]. The specific variants of the schemes implemented in JutulDarcy are based on the MRST module used in [40, 41]. In addition, a family of weighted essentially non-oscillatory (WENO) schemes for general unstructured meshes from [42] is supported.

We can demonstrate this functionality on a small model inspired by example 6.1.2 in [7]. The model is a small rectangular domain with an injector placed centrally at the top of the domain, and producers in the opposing corners.

 https://doi.org/10.1007/s10596-025-10366-6

As the material has a homogeneous permeability and porosity, the problem is symmetric and equal amounts of injected fluid should be produced in both producers. However, the mesh is significantly skewed towards the right producer, leading to significant grid orientation effects when solved by a standard two-point flux approximation. In addition, as we have used linear relative permeability curves, the fluid front will move through the domain as a contact discontinuity that is highly sensitive to numerical diffusion. The mesh and results are plotted in Fig. 2. We observe that switching to the consistent AvgMPFA scheme corrects the direction of the flow, but does not limit the smearing, while the opposing result holds for the WENO scheme. Combining both schemes improves both aspects of the solution, but comes at a cost of nearly tripling the total runtime of the simulation. Having alternative schemes available in a simulator is important to be able to assess the quality of grids and the accuracy of a discretization for a certain physical process, even if it is not always feasible to directly use the more advanced schemes on a large and complex reservoir model and obtain results in a timely manner.

5.4 Quarter-five-spot with sensitivities

The next example is a synthetic, conceptual CO_2 injection model that builds upon the previous example by adding more realistic fluid properties and a set of permeability barriers that impede flow. Gas is injected in the lower-left corner at a constant rate, with fluids produced at constant bottom-hole pressure in the upper right corner. The domain is discretized into a Cartesian 500×500 mesh, with a total of 250,000 cells. The producer initially only produces water, but eventually the injected gas bypasses the barriers and breaks through. Figure 3 shows the permeability and Fig. 4 the gas front at different steps. The code for setting up the example is largely the same as in the introductory example and is for brevity not repeated here. Instead, we will examine how we can add a new constitutive law together with a corresponding set of parameters to an already existing model, and how to

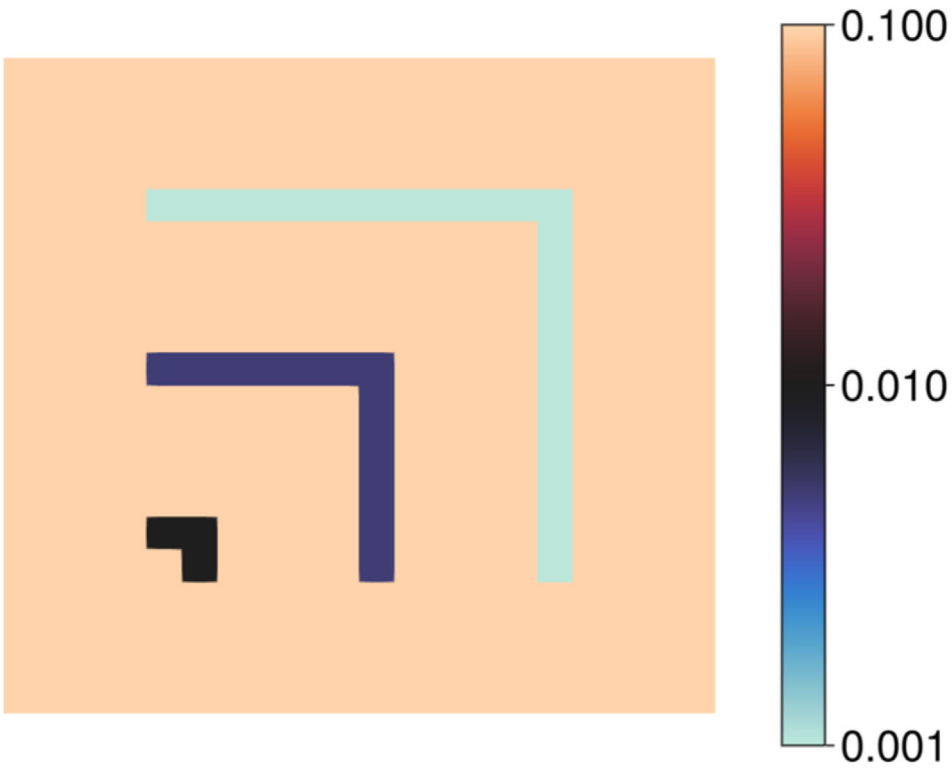


Fig. 3 Permeability, given in units of darcy, for the quarter-five-spot example with three low-permeable regions that impede flow

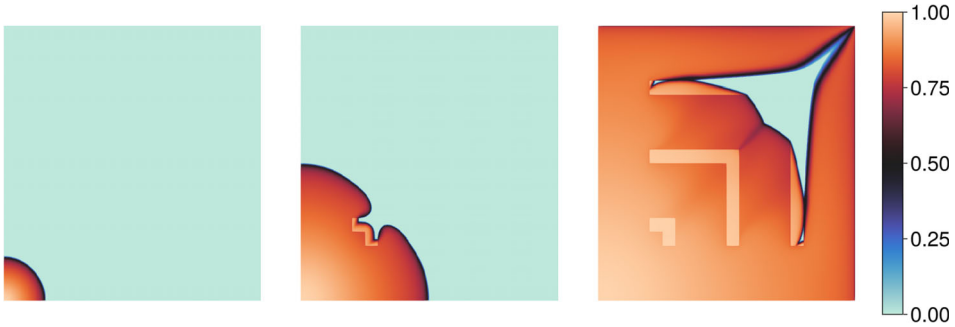


Fig. 4 Progression of gas saturation for three different time-steps: Just after injection start, after the front has reached the first barrier and at gas breakthrough in the upper right producer

then define an objective function and finally compute sensitivities with the adjoint method with respect to all input parameters.

Starting from an already specified water–gas model, we want to add a simple Brooks–Corey relative permeability on the form $k_{rw} = S_\alpha^n$, $k_{rg} = S_\alpha^m$, where the phase exponents n and m can vary from cell to cell. In Jutul parlance, this is called adding a secondary variable. We subtype the abstract relative permeability type and add an update function that is called when new values are needed by the simulator:

```
1  import JutulDarcy: AbstractRelativePermeabilities
2  struct MyKr <: AbstractRelativePermeabilities end
3  @jutul_secondary function update_my_kr!(vals, def::MyKr, model,
       Saturations, KrExponents, cells_to_update)
4      for c in cells_to_update
5          for ph in axes(vals, 1)
6              S_α = max(Saturations[ph, c], 0.0)
7              n_α = KrExponents[ph, c]
8              vals[ph, c] = S_α^n_α
9          end
10     end
11 end
```

Secondary variables are updated through a standard interface, in which `vals` is the array of values to update, `def` is the type instance of the secondary variable, `model` the discretized model, and the last argument is the list of entities to be updated. Julia allows a function to be specialized on multiple input arguments, and we could have specialzied this evaluator in the array storage type used for `val`, and the type of system `model` is defined for, for instance if a we wanted a different definition when this relative permeability function was run for a black-oil model with a certain type of AD array type. Arguments in between these are special in that their names in the function declaration are assumed to be valid variables or parameters for the model itself. The `jutul_secondary` annotation is a macro that parses the function arguments and stores the dependencies of the function. We observe that although this code is a fairly straightforward loop without many type annotations, evaluation during a simulation will be highly efficient and provide all required derivatives if the input variables are of the right AD type.

Now that we have declared a function that assumes the named `KrExponents` variable exists in evaluation, we should also define a type that can implement the cell-wise exponent. We do this by subtyping the abstract type for phase variables and add a default value of 2.0:

```
1  import JutulDarcy: PhaseVariables
2  struct MyKrExp <: PhaseVariables end
3  Jutul.default_value(model, ::MyKrExp) = 2.0
```

Given both a new secondary variable in the form of a new relative permeability and a new parameter in the form of the corresponding exponents, these can be set in the model, and we can plot the *variable graph* afterwards (see Fig. 5):

 https://doi.org/10.1007/s10596-025-10366-6

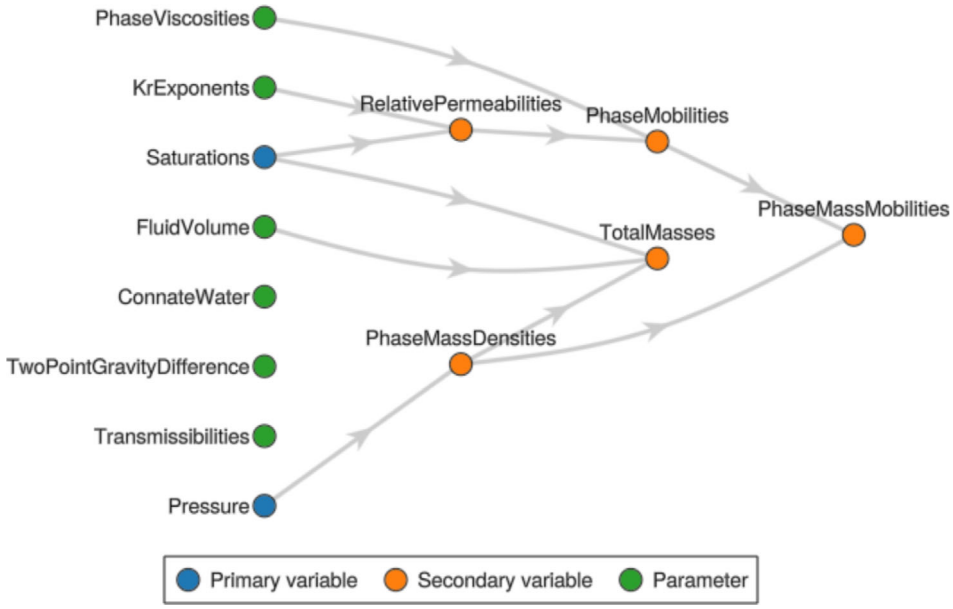


Fig. 5 The variable graph for the modified quarter-five-spot example. We see the relationship between primary variables, secondary variables, and parameters for our custom model. Note that the `KrExponents` parameter node has been added, and the `RelativePermeabilities` node has been given a new implementation that uses the new parameters. Note that all variables and parameters can be used when the model equations are evaluated

```
set_parameters!(rmodel, KrExponents = MyKrExp())
replace_variables!(rmodel, RelativePermeabilities = MyKr())
Jutul.plot_variable_graph(rmodel)
```

The variables and parameters in the model form a directed graph that indicates the dependency of each secondary variable on the other variables and parameters. Each node has both a name (e.g., `RelativePermeabilities`) and an implementation (e.g., `MyKr`) that can be swapped with an alternative one at runtime. This graph is dynamic in that new nodes can be added as needed, and connections between nodes are implicitly added by the evaluator functions. The simulator only requires that the graph does not contain loops and that all named dependencies are present in the graph before simulation. Code for efficient evaluation of the current graph is automatically compiled at the beginning of a simulation.

We next initialize the parameters of the model, and modify the exponents to have different values for the liquid and gas phases, and alternative values in the low-permeable barriers.

```
prm = setup_parameters(model)
exponents = prm[:Reservoir][:KrExponents]
for (cell, rtype) in enumerate(rock_type)
    if rtype == 1
        exponents[1, cell] = 2
        exponents[2, cell] = 3
    else
        exponents[1, cell] = 1
        exponents[2, cell] = 2
    end
end
```

Here, `rock_type` is a cell-wise indicator that has values larger than 1 in the barriers. We finally pack the complete setup into a simulation case and create a simulation result:

```
case = JutulCase(model, dt, forces, parameters = prm, state0 = s0)
result = simulate_reservoir(case, output_substates = true)
ws, states = result # To wells and states
```

Once the forward simulation is complete, we can see the effect of both the low permeability and the different relative permeability exponents in the gas saturations shown in Fig. 4, indicating that we have successfully added a new constitutive relationship to the simulator.

The next step is to compute the sensitivities of a quantity of interest with respect to the model parameters. In our setup, CO_2 may be produced by the pressure-support well in the upper right corner, which is undesirable in a storage scenario. We are therefore interested in finding out which model parameters influence the amount of CO_2 gas produced and the magnitude of their impact. We thus define the objective function to be the sum of the produced gas volume, normalized by the total volume of gas injected into the reservoir at the same conditions. The objective functions in `Jutul.jl` are by default defined by a sum over all time steps, and the user only needs to provide a function to calculate each term of the sum. We write the objective function by making use of a utility that computes the gas rate for a given well:

```
pv = pore_volume(model, prm)
total_time = sum(dt)
inj_rate = sum(pv)/total_time
import JutulDarcy: compute_well_qoi
function objective_function(model, state, Δt, step_i, forces)
    T = SurfaceGasRateTarget
    grat = compute_well_qoi(model, state, forces, :Producer, T)
    return Δt*grat/(inj_rate*total_time)
end
```

As we saw earlier, the code is written in a flexible scripting-like syntax, where variables like `inj_rate` and `total_time` defined before the function are implicitly brought into the objective function.

Sensitivities can in this case be computed by a high-level utility function that solves the adjoint system and simultaneously computes the chain rule of the numerical parameters (e.g., transmissibility and pore volumes) with respect to the parameters of the reservoir model (e.g., permeability, porosity, and geometry):

```
import JutulDarcy: reservoir_sensitivities
grad = reservoir_sensitivities(case, result, objective_function)
```

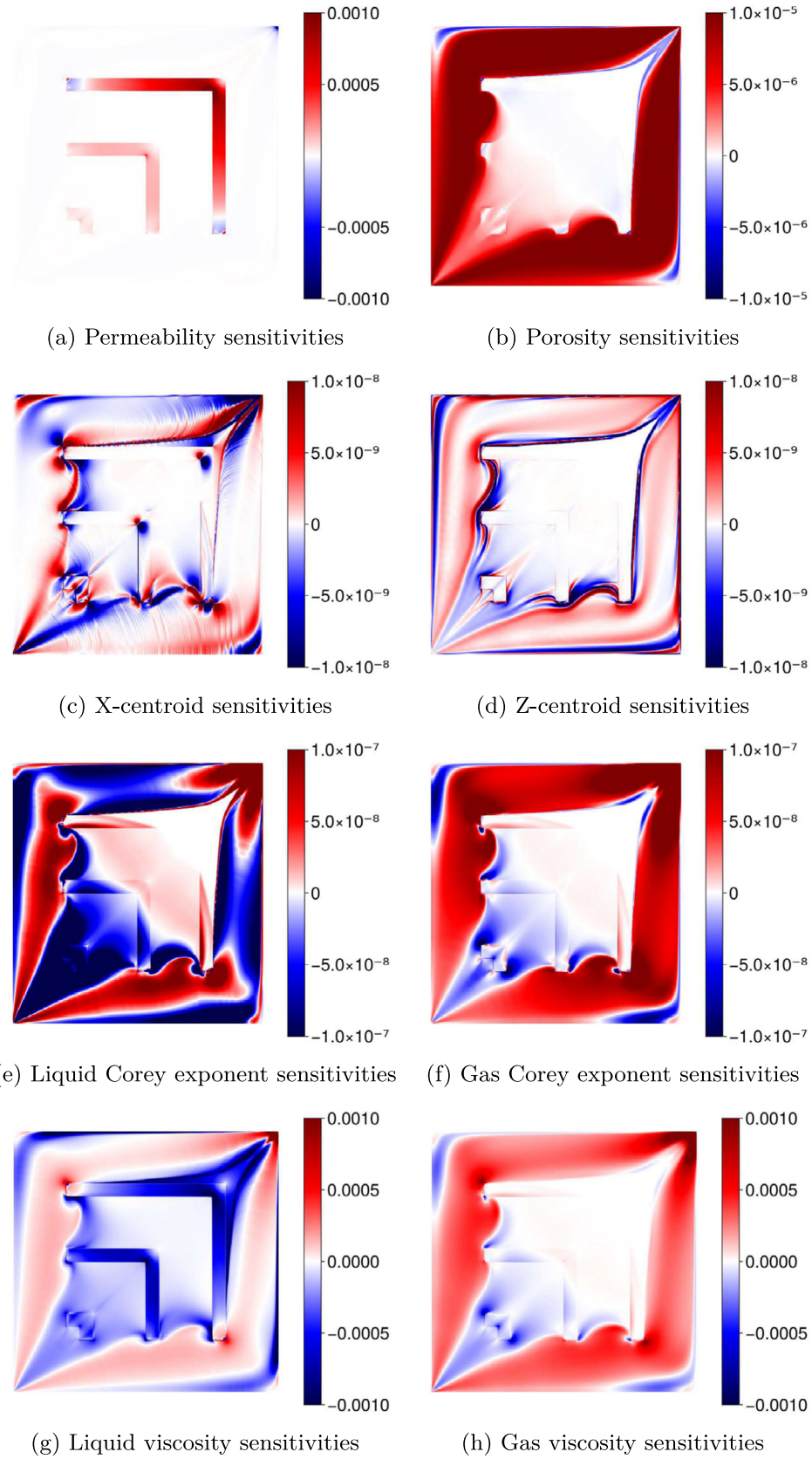
We plot a subset of the nonzero sensitivities in Fig. 6. Due to the fixed injection rate, the permeability of the high-permeable background rock type does not significantly influence the amount of gas produced, but any increase in permeability for the barriers will clearly increase the amount of gas produced. The opposite situation is seen for the porosity, where the value of the background matrix has a strong positive gradient; adding more pore space to hold fluids increases residence time and delays breakthrough to the producer.

The effects of the other parameters are more less obvious: Sensitivity with respect to the x-component of the cell centroid has a complex pattern. For the z-coordinate, the density difference between gas and water means that gravity would take effect if the domain were to be angled away from the xy plane. We also obtain sensitivities with respect to our custom Corey-exponent parameters, with the trend that increasing the liquid exponent reduces gas production, while increasing the gas exponent has the opposite effect. The final set of sensitivities are the fluid viscosities, for which an increase of either viscosity in the high-flow channel would increase the gas production. We finally remark that fluid viscosities were parameters in this case, but the fluid viscosity could easily be switched over to secondary variables in much the same way we added the new relative permeability variable and corresponding parameter.

5.5 Compositional validation

This case is a small compositional problem inspired by the examples in [43]. A 1D reservoir of 1,000 meters length is discretized into 1,000 cells. The model initially contains a mixture made up of 0.6 parts C_{10}, 0.1 parts CO_2, and 0.3 parts C_1 by moles at 150 °C and 75 bar pressure. Wells are placed in the leftmost and rightmost cells of the domain, with the leftmost well injecting pure CO_2 at a fixed bottom-hole pressure of 100 bar and the other well

 https://doi.org/10.1007/s10596-025-10366-6



(a) Permeability sensitivities (b) Porosity sensitivities

(c) X-centroid sensitivities (d) Z-centroid sensitivities

(e) Liquid Corey exponent sensitivities (f) Gas Corey exponent sensitivities

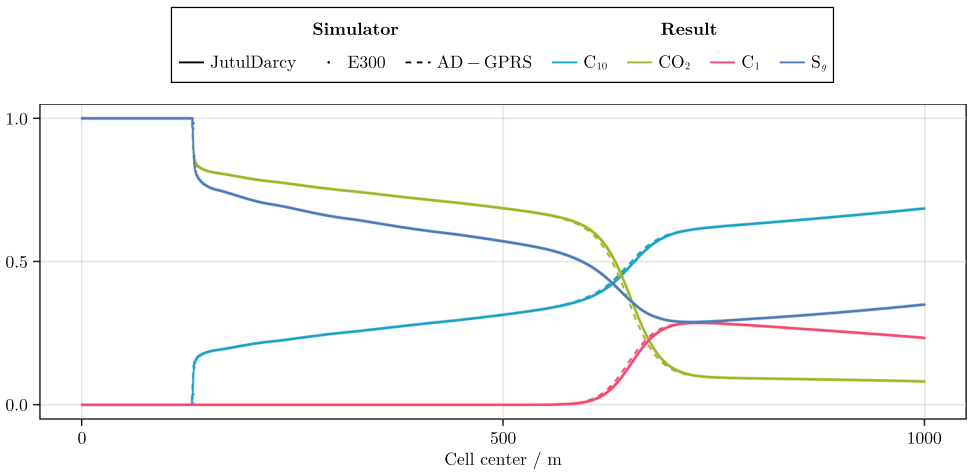
(g) Liquid viscosity sensitivities (h) Gas viscosity sensitivities

Fig. 6 Sensitivities of the normalized gas production rate in the producer in the upper right corner, with respect to a selection of spatially distributed parameters for the modified quarter-five-spot example

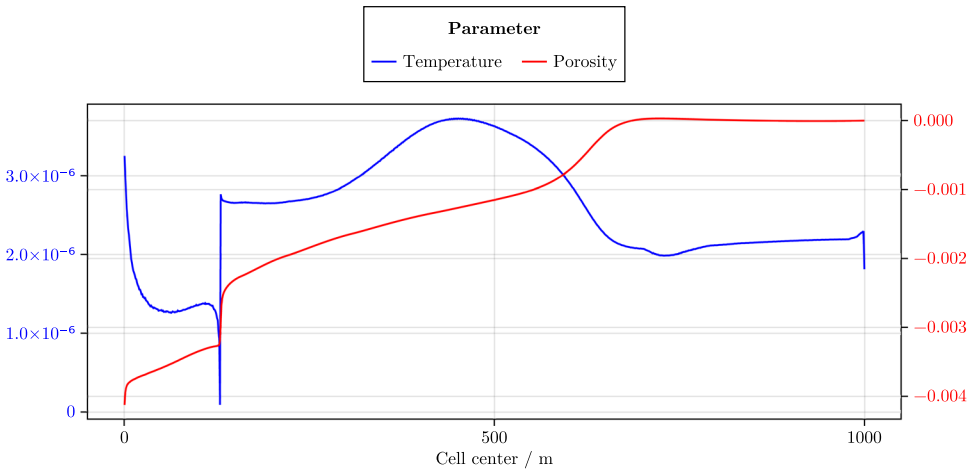
producing at 50 bar. The model is isothermal and contains a phase transition from the initial two-phase mixture to single-phase gas as injected CO_2 eventually displaces the resident fluids. For further details on this setup, see [44].

The problem may appear quite simple, but the case has a few intentionally difficult aspects: i) the wells operate at bottom-hole pressure and the amount of gas injected is highly sensitive to the predicted pressure of the model; ii) mixing of CO_2 into the liquid phase changes the viscosity and density and this can quickly alter the amount of CO_2 injected via the pressure; and iii) phase behavior and properties are all predicted by the equation-of-state with only molecular weights and critical properties being given as inputs. As we are going to compare simulators directly and want to avoid differences in internal timesteps, we prescribe a ramp-up of 15 steps that start at 0.01 day and double until they reach 3 days. This is followed by 12 steps of 5 days and 500 additional steps of 15 days each. These timesteps were selected so that Eclipse 300 does not subdivide timesteps, making a direct comparison possible.

The results are plotted for step 250 in Fig. 7a, where all three simulators give nearly identical results, with Eclipse 300 and JutulDarcy being indistinguishable. JutulDarcy uses 34.8 seconds and 1576 Newton iterations to simulate this case with default settings and a single single-thread on a fairly old Intel Xeon E3-1275 CPU.



(a) Mole fractions and gas saturation at step #250.



(b) Gradients of the mean gas fraction at step #250.

Fig. 7 Results for the compositional validation example. The solution from the three different simulators JutulDarcy, Eclipse 300 (E300), and AD-GPRS show excellent agreement. We also plot the sensitivities of the mean gas saturation in the above plot with respect to temperature and porosity

Out of this time, 84.1% is property evaluations that are dominated by the flash calculations. To speed things up a bit, we enable the fast_flash option that implements several techniques from [45] to reduce the cost of stability testing and successive two-phase flashes under similar conditions. Although there are no changes in the linear and nonlinear iterations, the total runtime is reduced to 21.9 seconds. In either case, the time spent on variable updates, assembly of equations and automatic differentiation accounts for approximately 1.4 seconds. For comparison, running the same case in AD-GPRS on the same CPU uses approximately 9.5 seconds for discretization and 48.0 seconds for property evaluations. Running the same case in `jutuldarcy.jl` on the more modern Ryzen 9 CPU, the single-thread runtime is 18.2 without the flash option and 9.3 seconds with. If run with 10 threads, the runtimes drop to 5.3 and 3.3 seconds, respectively, with 0.4 seconds spent in assembly.

We next pick the average gas saturation of our selected step as an objective function and compute gradients. This is done by returning a zero value for the objective if the step is not equal to 250. In this case, we load the standard statistics library to get a mean function. The mean function is trivial to write by hand, but using the standard library demonstrates that the objective functions can contain general Julia code that was not written with AD or the adjoint method in mind:

```
using JutulDarcy
case = setup_case_from_data_file("VALIDATION_1D.DATA", fast_flash = true)
result = simulate_reservoir(case, output_substates = true)
import Statistics: mean
import JutulDarcy: reservoir_sensitivities
function objective_function(model, state, Δt, step_i, forces)
    if step_i != step_to_plot
        return 0.0
    end
    sg = state.Reservoir.Saturations[2, :]
    return mean(sg)
end
grad = reservoir_sensitivities(case, result, objective)
```

Figure 7b shows the sensitivity with respect to temperature and porosity. The gradient with respect to porosity is close to zero beyond the gas front but negative in the rest of the domain. Increasing the porosity in cells that are passed by the gas front early on would both slow down the gas front, making it smaller, and ensure that more gas is contained in cells with a large pore volume. As the gas saturation is independent of the size of the cell, the objective function would be reduced. For the temperature gradient, it appears that increasing the temperature in the two-phase zone where miscibility occurs would create more gas, a prediction that is consistent with the expectation that the amount of gas increases for higher temperature. We finally note that temperature was a parameter for this model since the problem was isothermal, while for other models, temperature could either be a solution variable or a secondary variable.

5.6 Norne with sensitivities

The Norne oil and gas field from the Norwegian Sea serves as the next example. Among the different models representing this field,we use the one hosted by the Open Porous Media (OPM) on GitHub [8]. The model has a fairly complex $46 \times 112 \times 22$ corner-point grid, consisting of 44,431 active cells. Notable features include an intricate historical schedule with both gas and water injection, 36 active wells, calculation of initial state from hydrostatic equilibrium and fluid contacts, endpoint scaling of relative permeabilities in every cell, dissolved gas and vaporized oil, and scaling of capillary pressure to honor initial water saturation. Although the model itself is not large by modern standards, it is complex in the sense that taking the input file and producing correct results is difficult. Figure 10b shows the porosity plotted on the processed mesh together with the wells.

The model can be run directly in `JutulDarcy.jl`, except that threshold pressures and some hysteresis effects (vaporized oil and capillary pressure hysteresis) are neglected. To make direct comparison with OPM Flow easier,

we made a new input file that omit the unsupported features, with hysteresis being removed altogether even if JutulDarcy does support relative permeability hysteresis. We compare producer phase rates and injector bottom-hole pressures in Fig. 13. The match is generally good, especially considering that the simulators have different well models and can use different internal timesteps to advance the solution.

The next step is to calculate parameter sensitivities for this model. The procedure is more or less the same as for the previous sensitivity examples. We first simulate the model, identify which wells are producers, and let the objective function be the sum of producer gas rates. Loading the corner-point model, simulating with high computational efficiency, and calculating the full set of parameter sensitivities is 20 lines of code in total:

```
using JutulDarcy, HYPRE
case = setup_case_from_data_file("NORNE_NOHYST.DATA")
result = simulate_reservoir(case, output_substates = true)
prod = []                    # Find producers for use in objective
for (name, wsol) in pairs(result.wells)
    if sum(wsol[:rate]) < 0 # Negative rate -> producer
        push!(prod, name)
    end
end
total_time = sum(case.dt) # Normalize objective by total time
import JutulDarcy: compute_well_qoi, reservoir_sensitivities
function objective(model, state, Δt, step_i, forces)
    grat = 0.0
    T = SurfaceGasRateTarget # Get out gas rate
    for w in prod
        grat += compute_well_qoi(
        model, state,
        forces, w, T)
    end
    return Δt*grat/total_time # Total gas production, normalized by time
end
grad = reservoir_sensitivities(case, result, objective)
```

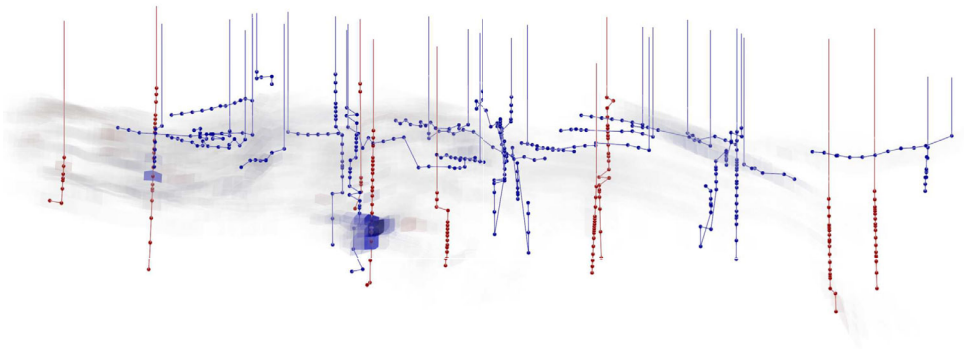
The forward simulation used 474 substeps to resolve the 247 report steps of the simulation schedule. We store the output of all ministeps to increase the accuracy of the adjoint solution. Executing the adjoint solve, including 474 linear solves, setting up each step from the previous solution, and differenting to obtain chain rules from input parameters to numerical parameters, takes approximately four minutes in a single threaded, which is comparable to the runtime of the forward simulation. Figure 8 shows plots of sensitivities with respect to porosity and net-to-gross.

We can examine the model a bit more in detail to put the output of the adjoint solve into context: The Norne model altogether has in total 133,395 primary variables, out of which 133,251 belong to the reservoir model (44,417 cells with three degrees of freedom each) while the remainder comes from the 36 wells and the facility model. The model has a total of 1,065,934 numerical parameters, and a single adjoint pass thus gives us gradients of the gas production with respect to all of them. Table 6 in the Appendix details all numerical parameters for the model, while Table 7, also in the Appendix, lists the input parameters for the model. We note that as this is a more complex model than the ones seen in the previous example, since there are additional parameters present. We automatically get gradients with respect to these new parameters, e.g., to the 16 relative-permeability scaling points in each cell. Just like the Corey exponents in the first sensitivity example, the relative-permeability scaling points are dynamically added to the simulator during parsing, and gradients are automatically computed because the code has a systematic approach to how parameters are declared along with adjoint capability and full support for automatic differentiation.

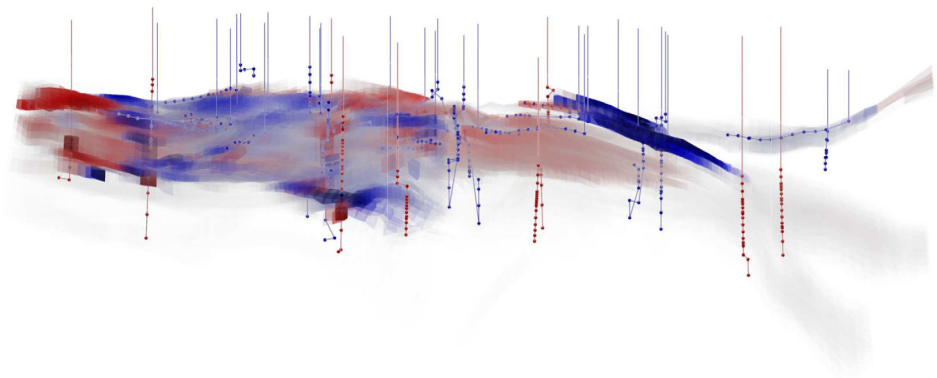
5.7 Performance in serial and parallel

We conclude the examples by examining solver performance. Our focus here is on the forward simulations, which typically are the most expensive part of a differentiable workflow. The solution of the adjoint equations make use

 https://doi.org/10.1007/s10596-025-10366-6



(a) Gradient with respect to net-to-gross



(b) Gradient with respect to porosity

Fig. 8 Scaled sensitivities of total gas production in the Norne model with respect to two chosen parameters. Red indicates that a small increase in the parameter would increase total gas production, while blue indicates that a small increase would decrease the total gas production. Visualizing the sensitivities becomes difficult for a complex 3D meshes than for structured 2D meshes, so the plots use dynamic transparency proportional to the absolute value of the sensitivity of each cell

of many of the same routines, and benchmarking a forward simulation is thus a good indicator of performance for the full simulation. Simulations are all performed with default settings for timestep selection, variable change limits, and convergence criteria. As with most reservoir simulators, models can have their runtime reduced by changing the parameters on a case by case basis. In all cases, the compressed-row-sparse (CSR) matrix format was used together with `HYPRE.jl` loaded for access to BoomerAMG.

In the following, we will in particular be concerned with measurables that are grouped under AD. This label covers all part of the computation in which automatic differentiation is used. It is not the overhead of automatic differentiation itself, rather, it is the cost of computing both the values and derivatives of all secondary variables, equations, and coupling terms with respect to all primary variables of the model. We emphasize that a simulator without AD would nevertheless have to compute the same values and derivatives, but would do so with hand-written code instead of AD. For instance, for a compositional model, AD would include the flash calculations, which are often a bottleneck in any simulator due to the vapor–liquid equilibrium calculations that do not make use of AD.

Serial performance We run the solvers in serial mode on all the models summarized in Table 3, which includes models we have already seen so far, a few other examples of open models, as well as a set of real-field models, named using single letters, for which results from `JutulDarcy.jl` have been matched against commercial simulators. Model sizes range from 300 cells for SPE1 to approximately two million cells for the Sleipner model. Statistics on runtime performance are show in Table 4.

Even for models with approximately four million degrees of freedom, the total cost of a linearization is still less than one second. All models are dominated by the linear solve, where the most significant contributor is the cost of calling the BoomerAMG preconditioner. Performance, both in terms of cost per linearization and Newton solve, and in terms of total runtime is competitive with commercial offerings, with some models performing significantly better in `JutulDarcy.jl`. This clearly demonstrates that a simulator based on automatic differentiation is sufficiently fast, without compromising the flexibility needed for the examples in this paper.

The variations among different simulators that make use a fully-implicit discretization with a TPFA-SPU type flux largely comes down to the type of linear solver used, the convergence criterion and the time-stepping algorithm, parameters that are adjusted on a case-by-case basis by experienced engineers who are familiar with any given simulator. As most models in this table are publicly available, we encourage interested readers to perform this test on their own hardware and any simulators of interest.

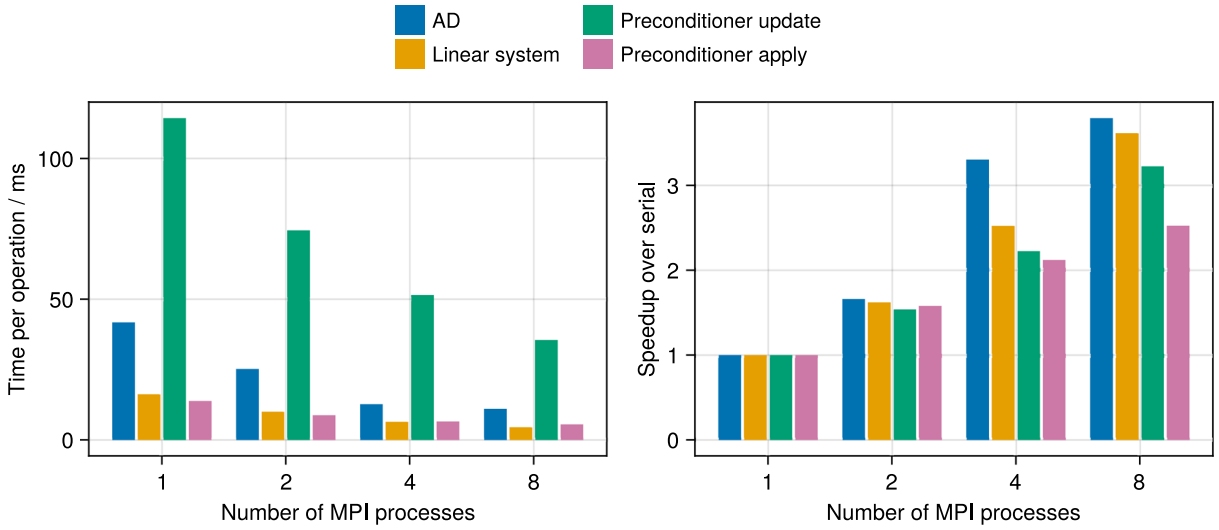
Parallel performance `JutulDarcy.jl` supports both the use of threads and MPI for parallel execution on CPU. For many models, the largest cost to total runtime is the linear solver, for which MPI is the most common parallelization model where a problem is subdivided between multiple processes. We therefore perform a strong scaling test on the OLYMPUS, B, and Sleipner models. The OLYMPUS model and B models are the same models as described earlier, and the Sleipner model is a CO_2 injection case with just under two million cells, with approximately ten times the number of cells as that of OLYMPUS. In the Sleipner model, supercritical CO_2 is injected in the lowest part of the formation, and the upwards migration is limited by the low-permeable explicitly gridded shales (light blue in plot) between each region of high permeability sand (light orange in plot). This version of the Sleipner model uses a simple immiscible two-phase fluid model that runs in one hour in serial.

Figure 9 summarizes the scaling of the operations involving AD, linear system assembly, setup and application of the preconditioner. The test is performed on a single 16-core CPU, which means that memory bandwidth is the main limitation for parallel scalability of a reservoir simulator. We observe that AD and assembly scale to roughly four times speedup for both cases on 8 processes, while the application of the preconditioner is less improved by additional cores, especially for Sleipner.

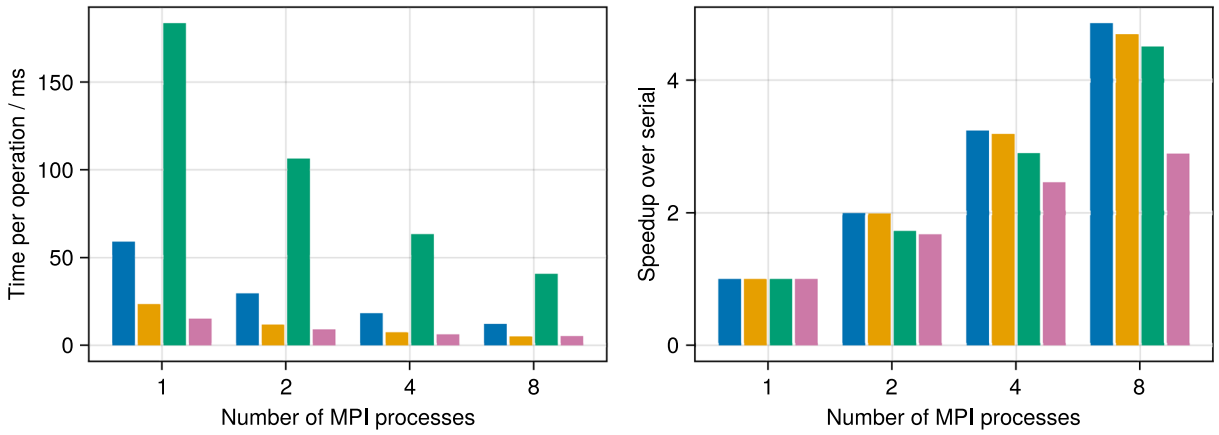
The Sleipner model seen in Fig. 10d is known to be difficult for linear solvers, as the thin explicitly gridded shales contribute to very ill-conditioned linear systems. The total speedup of the simulation itself is 3.2 for OLYMPUS and 2.0 for Sleipner for 8 processors. For Sleipner, the main reason for the low degree of speedup, even though individual operations scale quite well, is that the 8 processor case uses 50% more linear iterations than the serial case, likely due to the accuracy of the ILU(0) preconditioner being reduced as the number of processes increase on this model. As the size of the Sleipner case appears to be limited by memory bandwidth for the linear solver, it is a prime candidate for GPU acceleration. Table 5 compares performance of a serial result against 10 CPU threads and a linear solver running on the GPU, where classical AMG on GPU in hypre is compared against classical AMG in the AMGX library. The block-ILU(0) preconditioner comes from CuSPARSE. Iteration numbers are very similar between GPU and CPU as the algorithms have been adjusted to be similar for a direct comparison. The linear system is assembled on CPU and transferred in double precision using pinned memory to the device. We observe a total speedup of 3.84, which outperforms CPU by some margin, with the linear solver itself reaching a speedup of 4.57 including the cost of transferring the system to device memory. From these numbers it is not clear how significant the benefit of porting the assembly code itself to GPU would be, as the assembly gets reasonable scaling on CPU, but is embarrassingly parallel and limited by memory bandwidth.

Further testing on multi-processor setups with higher memory bandwidth is required to assess the scalability of the implementation for larger models, but we note that a reduction in runtime with a factor 2–3 is achievable on a consumer-grade multicore CPU, with additional speedup being available if a CUDA-capable GPU is present. Irrespective of the number of processes, the cost of all linearization steps that make use of AD are similar to that of a single application of the CPR preconditioner. This is a another indication that the use of AD throughout a reservoir simulator is not an obstacle to high computational efficiency.

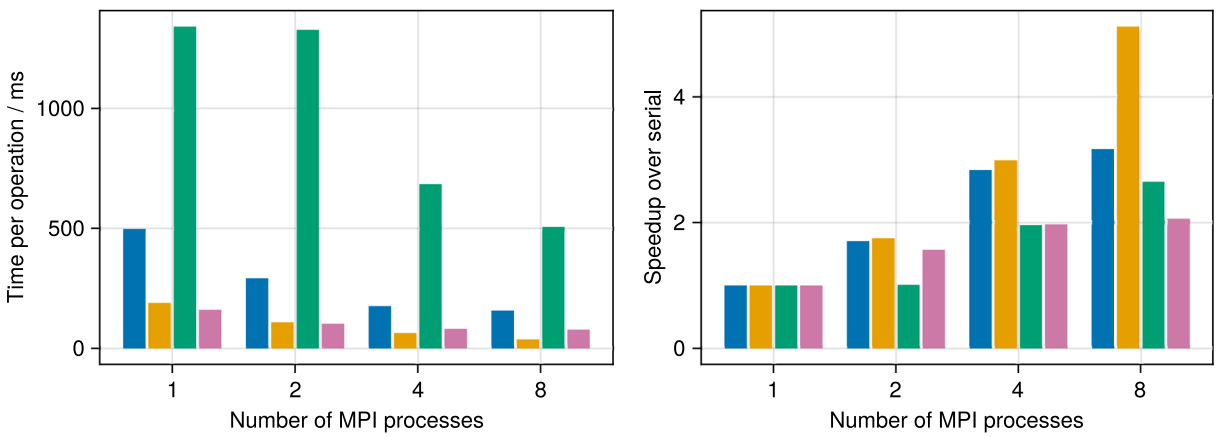
 https://doi.org/10.1007/s10596-025-10366-6



(a) Timing for the OLYMPUS model



(b) Timing for the real-field B model



(c) Timing for the Sleipner model

Fig. 9 Strong scaling of selected key operations in MPI parallel mode run on a single CPU for the OLYMPUS, B, and Sleipner models

6 Conclusions

We have demonstrated that a new fully differentiable reservoir simulator written in Julia can simultaneously achieve high flexibility and computational performance. The simulator is designed from the ground up to use automatic differentiation whenever possible, enabling highly complex and coupled models to be written on residual form, with automatic calculation of Jacobians and adjoint gradients. Tailoring the design to just-in-time compilation also makes the use of computational graphs that can be customized at runtime feasible. High computational efficiency reservoir simulation is possible with AD while simultaneously opening up many avenues for differentiable programming where reservoir simulation is one part of a complex workflow.

To support research into differentiable solvers for porous media flow, we clearly believe that the benefits of using Julia outweigh the disadvantages. The combination of a language syntax with little boilerplate, support for automatic differentiation, high performance, and multiple dispatch enables us to take a significant step forward to a flexible research code that can solve problems of industrial complexity with performance comparable to that of commercial offerings.

Appendix

The appendix contains figures and tables that are too large to be reasonably placed inline in the text.

Table 2 Examples of packages that extend or interact with `JutulDarcy.jl`

Module	Description
`GLMakie.jl`	OpenGL visualization [46].
`HYPRE.jl`	Adds the hypre library [47], in particular BoomerAMG [48] support for the CPR preconditioner.
`AMGCLWrap.jl`	Adds preconditioners from AMGCL [49].
`CUDA.jl`	Sparse linear algebra and array programming building blocks on CUDA GPUs [50, 51].
`AMGX.jl`	Algebraic multigrid on CUDA GPUs for CPR[52].
`PartitionedArrays.jl`	Array types for MPI-parallel scalable linear algebra [53].
`Clapeyron.jl`	Library of thermodynamic models [54], adds advanced equations of state, including PC-SAFT.
`Meshes.jl`	Computational geometry and meshing algorithms, support for additional mesh types and input formats.
`JutulDarcyRules.jl`	High-level differentiation rules [55] for coupling with seismic solvers and generative models for multiphysics inversion [56].

 https://doi.org/10.1007/s10596-025-10366-6

Table 3 A list of models used for performance testing in serial

Name	N_{ph}	N_c	Type	Cells	Wells	Report steps
SPE1	3	3	Black-oil	300	2	120
SPE9	3	3	Black-oil	9,000	26	90
Egg	2	3	Water-oil	18,553	12	135
OLYMPUS	2	2	Immiscible	192,750	18	20
Norne	3	3	Black-oil	44,417	36	247
Sleipner	2	2	Water-gas	1,986,176	1	171
A	3	7	Compositional	≈ 60,000	2	654
B	2	2	Oil-water	≈ 200,000	100+	≈ 400
C	3	3	Black-oil	≈ 150,000	250+	≈ 250
D	3	3	Black-oil	≈ 1,250,000	100+	≈ 100

Report steps refer to the number of points in time at which output is to be written, or where wells are given new controls and limits

Table 4 Performance numbers in serial for the range of test models

Name	Time steps	Linearizations (each)	AD	Linear solve	Total time
SPE1	126	506 (0.2 ms)	19.8 %	53.3 %	0.3 s
SPE9	96	403 (5.5 ms)	25.7 %	54.1 %	5.3 s
Egg	162	697 (4.7 ms)	11.9 %	74.7 %	14.1 s
OLYMPUS	93	731 (64.4 ms)	12.1 %	78.22 %	226.4 s
Norne	470	2510 (40.7 ms)	29.0 %	51.7 %	218.3 s
Sleipner	269	1672 (679.0 ms)	18.3 %	67.3 %	3,586.3 s
A	657	1968 (225.4 ms)	28.69 %	46.0 %	767.2 s
B	526	2282 (65.1 ms)	15.2 %	70.9 %	534.1 s
C	2117	18862 (130.4 ms)	16.41 %	72.14 %	9,748.1 s
D	896	7,259 (837 ms)	9.4 %	80.6 %	41,129.6 s

Note that the number of linearizations (residual and Jacobian) are annotated with the average time taken to perform one such linearization. The time spent in routines where automatic differentiation is in use is shown in the AD column. This number includes all property evaluations, flash calculations, calculation of source and coupling terms, and the AD step in the linearization of the governing equations for reservoir and wells

Table 5 Comparison of runtimes on serial CPU and 8 threads with the linear solve running on GPU

Solver	Newton its.	Linear its.	Linear solver	Other	Total	Speedup
Threads + GPU	1423	4292	528 s	405 s	933 s	1.0
Serial CPU	1403	4622	2414 s	1172 s	3586 s	3.84

The timings are given as total numbers for the entire simulation

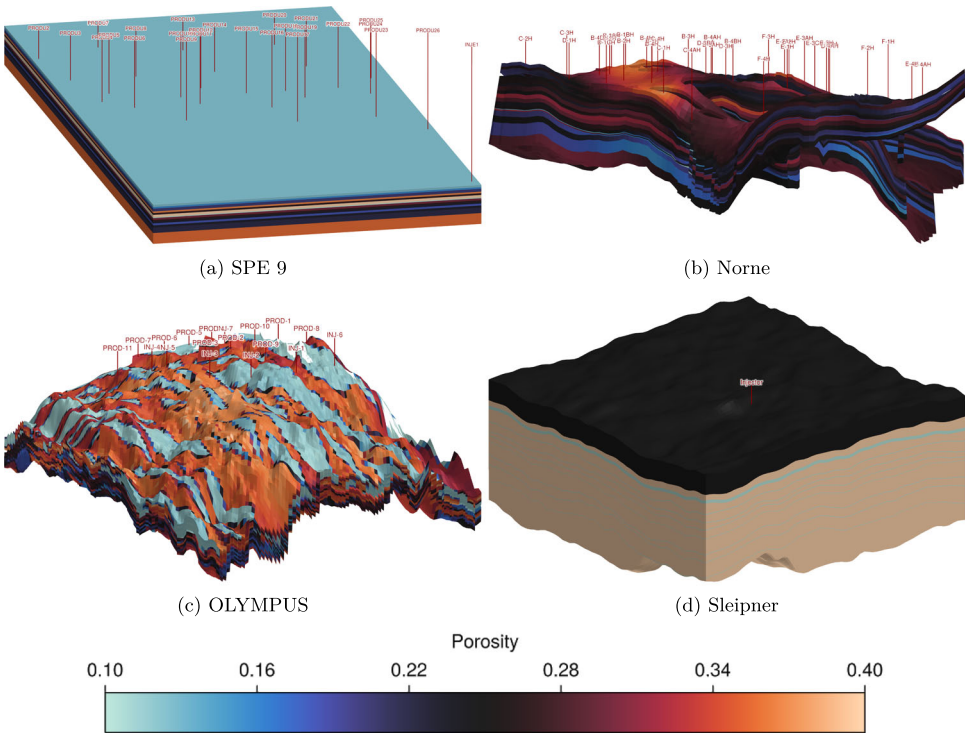
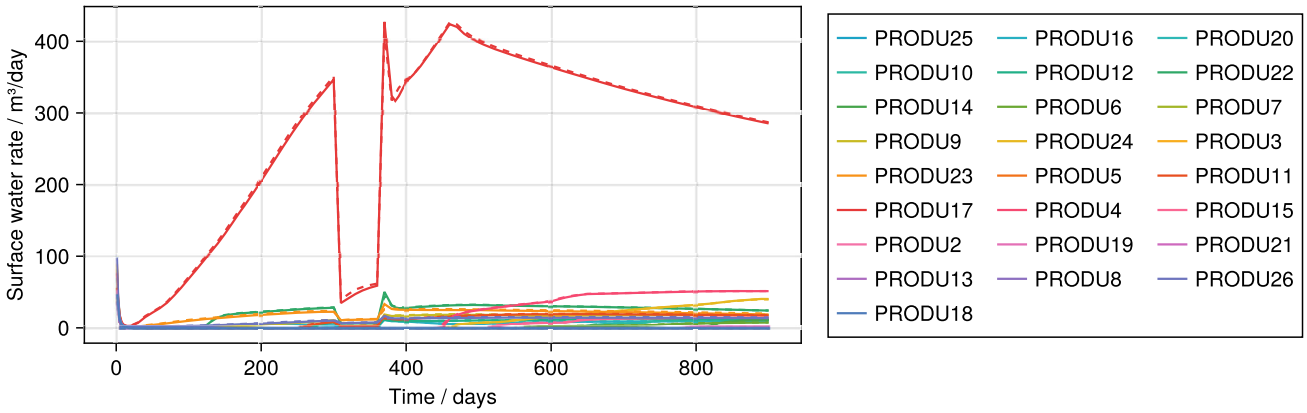
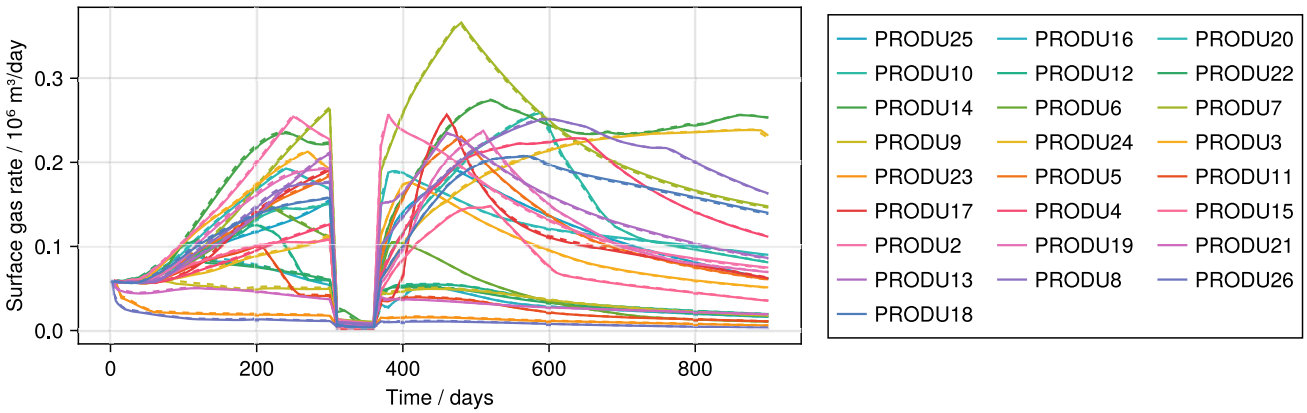


Fig. 10 Porosity of four publicly available corner-point test models: SPE 9, from the 9th SPE Comparative Solutions Project, OLYMPUS realization 1 from the ISAPP Field development optimization challenge, Norne and Sleipner, both from `opm-tests`

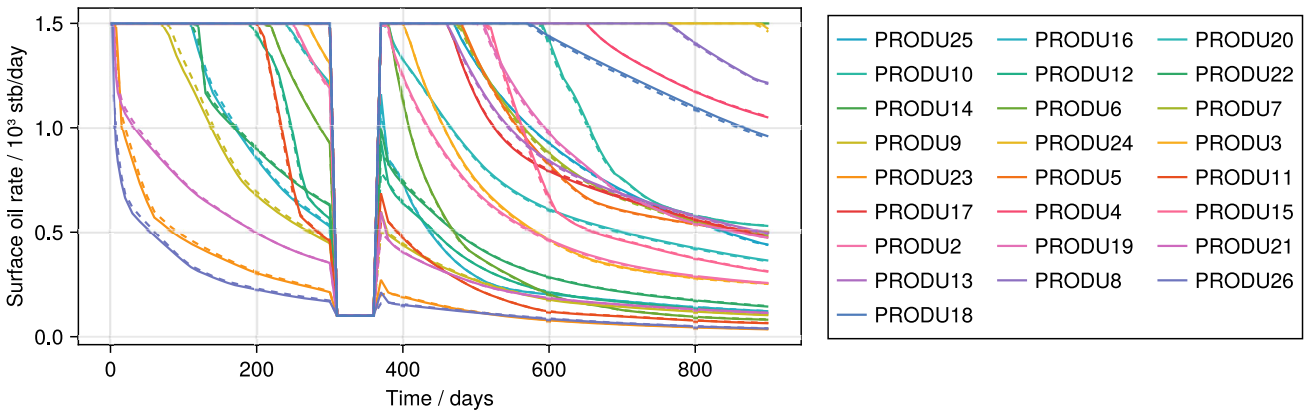
 https://doi.org/10.1007/s10596-025-10366-6



(a) Producer water rates

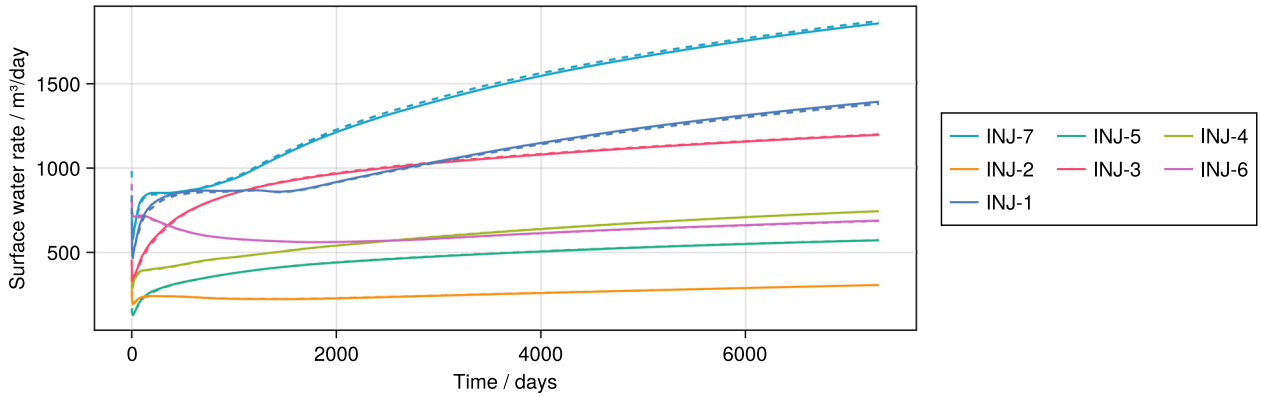


(b) Producer gas rates

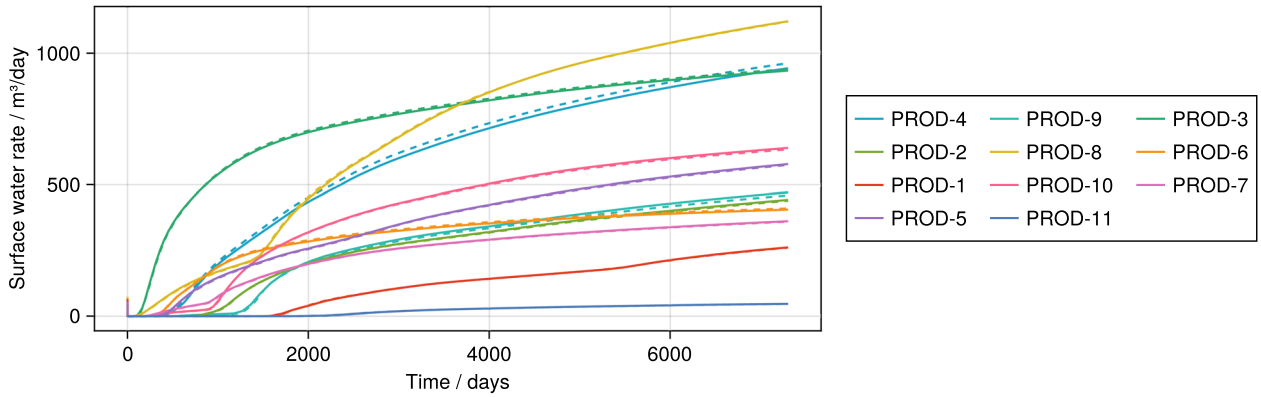


(c) Producer oil rates

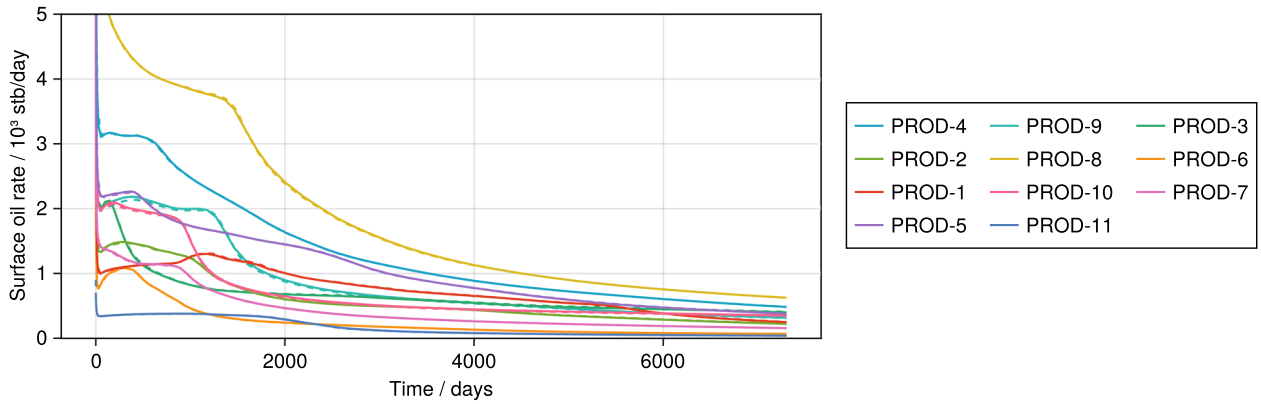
Fig. 11 Comparison between `JutulDarcy.jl` (whole lines) and Eclipse (dashed lines) on the SPE 9 corner-point model



(a) Injector bottom hole pressures



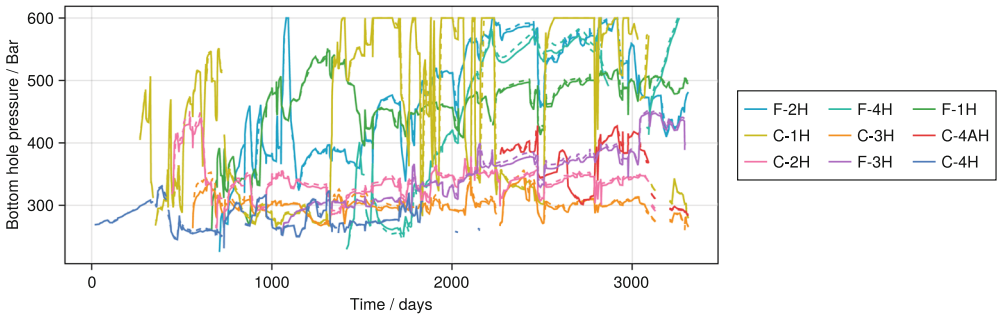
(b) Producer water rates



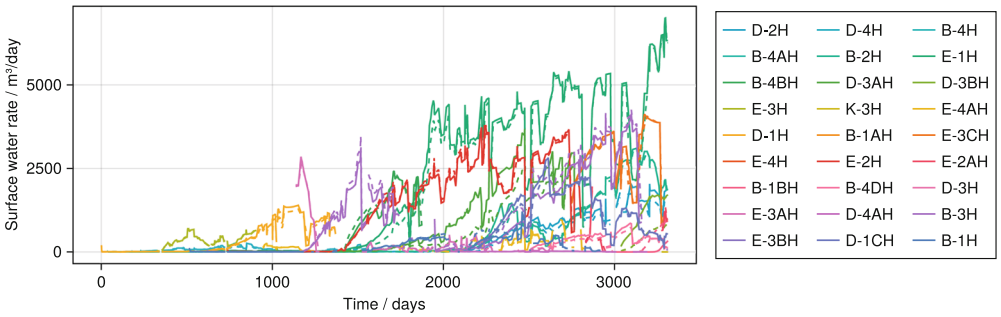
(c) Producer oil rates

Fig. 12 Well responses for `JutulDarcy.jl` (whole lines) compared to OPM Flow (dashed lines) for the Olympus 1 benchmark case

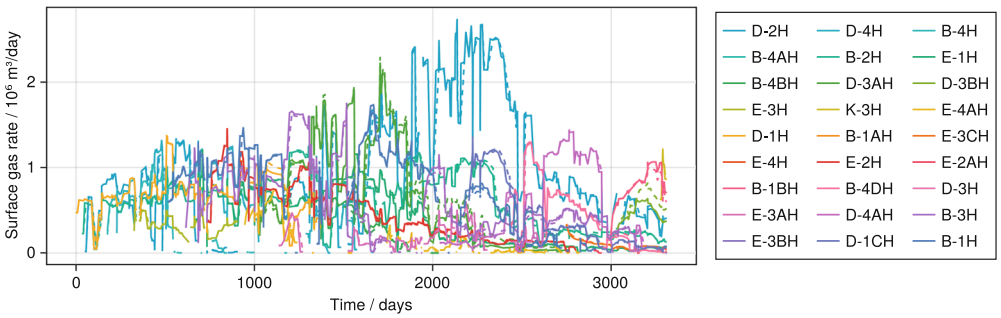
 https://doi.org/10.1007/s10596-025-10366-6



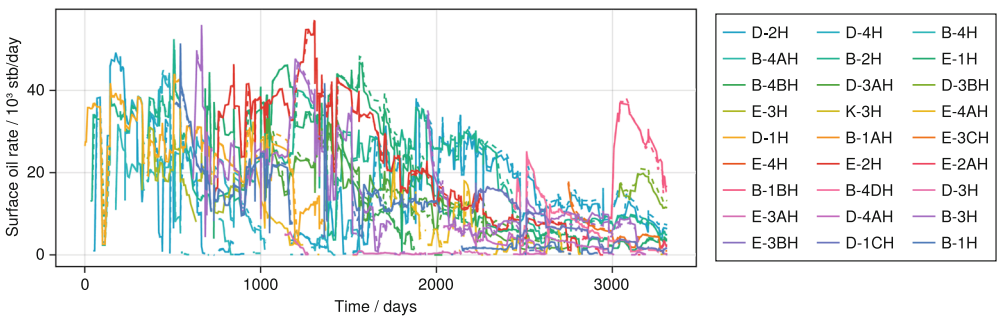
(a) Injector bottom hole pressures



(b) Producer water rates



(c) Producer gas rates



(d) Producer oil rates

Fig. 13 Well rates for `JutulDarcy.jl` (whole lines) and OPM Flow (dashed lines) for the Norne example. Both simulators read, process and initialize the mesh and initial state from the same file, where threshold pressures and hysteresis have been disabled from the original version of the case

Table 6 The *numerical* parameters of the Norne model as set up from an input file

Parameter	Count	Type	Note
Transmissibilities	132693×1	Faces	Discretizes potential difference
Gravity difference $g\Delta z$	132693×1	Faces	Discretizes bouyancy
Connate water	44417×1	Cells	Used in evaluation of k_r
Water k_r scalers	44417×4	Cells	End-points and maximum k_r
Oil-water k_r scalers	44417×4	Cells	–
Oil-gas k_r scalers	44417×4	Cells	–
Gas k_r scalers	44417×4	Cells	–
Pore-volume	44417×1	Cells	Pore space available to flow
Well indices	503×1	Perforations	Connection well and reservoir
Perforation Δz	503×1	Perforations	Depth from well to perforation
Top node volume	36×1	Well	Volume for standard well

These are numerical parameters associated with reservoir faces and cells, wells and their perforation that impact the forward simulation

Table 7 The *data* parameters of the Norne model as set up from an input file

Parameter	Count	Type	Impact
Areas	132693×1	Faces	Transmissibilities
Face centroids	132693×3	Faces	Transmissibilities
Normals	132693×3	Faces	Transmissibilities
Transmissibility multiplier	132693×1	Faces	Transmissibilities
Permeability	44417×3	Cells	Transmissibilities and well indices
Porosity	44417×1	Cells	Pore-volume
Net-to-gross	44417×1	Cells	Pore-volume, transmissibilities, well indices
Water k_r scalers	44417×4	Cells	Direct copy
Oil-water k_r scalers	44417×4	Cells	Direct copy
Oil-gas k_r scalers	44417×4	Cells	Direct copy
Gas k_r scalers	44417×4	Cells	Direct copy
Volumes	44417×1	Cells	Pore-volumes
Cell centroids	44417×3	Cells	$g\nabla z$, well perforation depth difference and transmissibilities

These parameters are set up from the input file and are used to initialize the numerical parameters. Some parameters are identical to their numerical counterparts and others are calculated through intermediate functions with complex internal logic. In this case, the input parameters are limited to the reservoir itself. The transmissibility multiplier incorporates faults and numerous other multipliers that are possible to input, for example through keywords like `MULTX`, `MULTRANZ` or `MULTREGT`

Acknowledgements The author would like to thank Knut-Andreas Lie, Halvor Møll Nilsen, Stein Krogstad and Junjian Li for many constructive discussions, as well as SINTEF colleagues who gave feedback on drafts of this manuscript.

Funding Open access funding provided by SINTEF. This work was supported in part by the Research Council of Norway through project number 308817, Digital wells for optimal production and drainage.

Data availability statement This paper makes use of a number of public datasets that are cited as they appear in the text. The source codes of the examples is available from the GitHub page together with links to the required data (https://github.com/sintefmath/JutulDarcy.jl). The exception is the performance examples that partially make use of non-public models that the author is unable to share. We believe that the inclusion of these models in the examples is important, as they are more complex than currently available public models.

Declarations

Competing Interest The authors declare that they have no competing interests that could have appeared to influence the work reported in this paper.

References

1. Hackstein, F.V., Madlener, R.: Sustainable operation of geothermal power plants: Why economics matters. Geotherm. Energy **9**, 1–30 (2021)
2. Singh, S.P., Ku, A.Y., Macdowell, N., Cao, C.: Profitability and the use of flexible CO2 capture and storage (CCS) in the transition to decarbonized electricity systems. Int. J. Greenhouse Gas Control **120**, 103767 (2022)
3. Baydin, A.G., et al.: Automatic differentiation in machine learning: A survey. J. Mach. Learn. Res. **18**(1), 5595–5637 (2017)
4. Abate, J., Wang, P., Sepehrnoori, K., Dawson, C.: Application of an automatic differentiation tool in the development of a compositional reservoir simulator. Commun. Numer. Methods Eng. **15**(6), 423–434 (1999)
5. Younis, R., Aziz, K.: Parallel automatically differentiable data-types for next-generation simulator development. In: SPE Reservoir Simulation Symposium (2007). Society of Petroleum Engineers
6. Krogstad, S., Lie, K.-A., Møyner, O., Nilsen, H.M., Raynaud, X., Skaflestad, B.: MRST-AD – an open-source framework for rapid prototyping and evaluation of reservoir simulation problems. In: SPE Reservoir Simulation Conference (2015). SPE
7. Lie, K.-A.: An Introduction to Reservoir Simulation Using MATLAB/GNU Octave. Cambridge University Press, Cambridge, United Kingdom (2019)
8. Rasmussen, A.F., et al.: The open porous media flow reservoir simulator. Comput. Math. Appl. 159–185 (2021). https://doi.org/10.1016/j.camwa.2020.05.014
9. Wang, Y., Voskov, D., Khait, M., Bruhn, D.: An efficient numerical simulator for geothermal simulation: A benchmark study. Appl. Energy **264**, 114693 (2020)
10. Hu, Y., Anderson, L., Li, T.-M., Sun, Q., Carr, N., Ragan-Kelley, J., Durand, F.: Difftaichi: Differentiable programming for physical simulation. arXiv:1910.00935 (2019)
11. De Montleau, P., Cominelli, A., Neylon, K., Rowan, D., Pallister, I., Tesaker, O., Nygard, I.: Production optimization under constraints using adjoint gradients. In: ECMOR X-10th European Conference on the Mathematics of Oil Recovery, p. 23 (2006). European Association of Geoscientists & Engineers
12. Kourounis, D., Durlofsky, L.J., Jansen, J.D., Aziz, K.: Adjoint formulation and constraint handling for gradient-based optimization of compositional reservoir flow. Comput. Geosci. **18**, 117–137 (2014)
13. Han, C., Wallis, J., Sarma, P., Li, G., Schrader, M.L., Chen, W.: Adaptation of the CPR preconditioner for efficient solution of the adjoint equation. SPE J. **18**(02), 207–213 (2013)
14. Tian, X., Delft, T., Voskov, D.: Efficient inverse modeling framework for energy transition applications using operator-based linearization and adjoint gradients. In: SPE Reservoir Simulation Conference (2023). OnePetro
15. Schlumberger: ECLIPSE: Technical Description, 2014.1 edn. Schlumberger, (2014). Schlumberger
16. Aanonsen, S.I., Nœvdal, G., Oliver, D.S., Reynolds, A.C., Vallès, B.: The ensemble kalman filter in reservoir engineering-a review. SPE J. **14**(03), 393–412 (2009)
17. Bezanson, J., et al.: Julia: A fresh approach to numerical computing. SIAM Rev. **59**(1), 65–98 (2017)
18. Grøstad, S.: Automatic differentiation in julia with applications to numerical solution of pdes. Master's thesis, NTNU (2019)
19. Anno, K., Moridis, G.J., Blasingame, T.A.: Jfts+h: A julia-based parallel simulator for the description of the coupled flow, thermal and geochemical processes in hydrate-bearing geologic media. In: SPE Reservoir Simulation Conference (2021). SPE
20. Jansen, J.D.: Adjoint-based optimization of multi-phase flow through porous media-a review. Comput. Fluids **46**, 40–51 (2011)
21. Wengert, R.E.: A simple automatic derivative evaluation program. Commun. ACM **7**(8), 463–464 (1964)
22. Neidinger, R.D.: Introduction to automatic differentiation and matlab object-oriented programming. SIAM Rev. **52**(3), 545–563 (2010)

23. Coleman, T.F., Verma, A.: The efficient computation of sparse jacobian matrices using automatic differentiation. SIAM J. Sci. Comput. **19**(4), 1210–1233 (1998)
24. Raynaud, X., Johansson, A., Nilsen, H.M., Watson, F., Clark, S.: BattMoTeam/BattMo: V0.3.0. https://doi.org/10.5281/zenodo.10633682
25. Lie, K.-A., Krogstad, S., Ligaarden, I.S., Natvig, J.R., Nilsen, H.M., Skaflestad, B.: Open-source MATLAB implementation of consistent discretisations on complex grids. Comput. Geosci. **16**, 297–322 (2012)
26. Møyner, O.: Faster simulation with optimized automatic differentiation and compiled linear solvers, pp. 181–230. Cambridge University Press, Cambridge, United Kingdom (2021). Chap. 6
27. Modular: Mojo – Programming language for all of AI. https://modular.com/max/mojo. [Online; Accessed 28 May 2024] (2023)
28. Bischof, C.H., Khademi, P.M., Buaricha, A., Alan, C.: Efficient computation of gradients and jacobians by dynamic exploitation of sparsity in automatic differentiation. Optim. Methods Softw. **7**(1), 1–39 (1996)
29. Daines, S.: SparsityTracing.jl. https://ithub.com/PALEOtoolkit/SparsityTracing.jl. [Online; Accessed 28 May 2024] (2021)
30. Revels, J., Lubin, M., Papamarkou, T.: Forward-mode automatic differentiation in Julia. arXiv:1607.07892 [cs.MS] (2016)
31. Knuth, D.E.: Literate programming. Comput. J. **27**(2), 97–111 (1984)
32. Nordbotten, J.M., Ferno, M.A., Flemisch, B., Kovscek, A.R., Lie, K.-A.: The 11th society of petroleum engineers comparative solution project: Problem definition. SPE J. **29**(05), 2507–2524 (2024)
33. Montoison, A., Orban, D.: Krylov.jl: A Julia basket of hand-picked Krylov methods. J. Open Source Softw. **8**(89), 5187 (2023). https://doi.org/10.21105/joss.05187
34. Møyner, O., Rasmussen, A.F., Klemetsdal, Ø., Nilsen, H.M., Moncorgé, A., Lie, K.-A.: Nonlinear domain-decomposition preconditioning for robust and efficient field-scale simulation of subsurface flow. Comput. Geosci. **28**(2), 241–251 (2024)
35. Killough, J.: Ninth SPE comparative solution project: a reexamination of black-oil simulation. In: SPE Reservoir Simulation Conference, p. 29110 (1995). SPE
36. Fonseca, R., Della Rossa, E., Emerick, A., Hanea, R., Jansen, J.: Overview of the olympus field development optimization challenge. In: ECMOR XVI-16th European Conference on the Mathematics of Oil Recovery, vol. 2018, pp. 1–10 (2018). European Association of Geoscientists & Engineers
37. Le Potier, C.: A nonlinear finite volume scheme satisfying maximum and minimum principles for diffusion operators. International Journal on Finite Volumes, 1–20 (2009)
38. Lipnikov, K., Shashkov, M., Svyatskiy, D., Vassilevski, Y.: Monotone finite volume schemes for diffusion equations on unstructured triangular and shape-regular polygonal meshes. J. Comput. Phys. **227**(1), 492–512 (2007). https://doi.org/10.1016/j.jcp.2007.08.008
39. Schneider, M., Gläser, D., Flemisch, B., Helmig, R.: Comparison of finite-volume schemes for diffusion problems. Oil & Gas Science and Technology - Revue d'IFP Energies nouvelles **73**, 82 (2018). https://doi.org/10.2516/ogst/2018064
40. Zhang, W., Al Kobaisi, M.: Cell-centered nonlinear finite-volume methods with improved robustness. SPE J. **25**(01), 288–309 (2020). https://doi.org/10.2118/195694-PA
41. Kobaisi, M.A., Zhang, W.: Nonlinear finite-volume methods for the flow equation in porous media, pp. 46–69. Cambridge University Press, Cambridge, United Kingdom (2021). Chap. 2
42. Lie, K.-A., Mykkeltvedt, T.S., Møyner, O.: A fully implicit WENO scheme on stratigraphic and unstructured polyhedral grids. Comput. Geosci. **24**(2), 405–423 (2020). https://doi.org/10.1007/s10596-019-9829-x
43. Voskov, D.V., Tchelepi, H.A.: Comparison of nonlinear formulations for two-phase multi-component eos based simulation. J. Petrol. Sci. Eng. **82**, 101–111 (2012)
44. Møyner, O., Tchelepi, H.A.: A mass-conservative sequential implicit multiscale method for isothermal equation-of-state compositional problems. SPE J. **23**(06), 2376–2393 (2018)
45. Rasmussen, C.P., Krejbjerg, K., Michelsen, M.L., Bjurstrøm, K.E.: Increasing the computational speed of flash calculations with applications for compositional, transient simulations. SPE Reserv. Eval. Eng. **9**(01), 32–38 (2006)
46. Danisch, S., Krumbiegel, J.: Makie.jl: Flexible high-performance data visualization for julia. J. Open Source Softw. **6**(65), 3349 (2021)
47. Falgout, R.D., Yang, U.M.: hypre: A library of high performance preconditioners. In: International Conference on Computational Science, pp. 632–641 (2002). Springer
48. Yang, U.M., et al.: Boomeramg: A parallel algebraic multigrid solver and preconditioner. Appl. Numer. Math. **41**(1), 155–177 (2002)
49. Demidov, D.: Amgcl: An efficient, flexible, and extensible algebraic multigrid implementation. Lobachevskii J. Math. **40**, 535–546 (2019)
50. Besard, T., Foket, C., De Sutter, B.: Effective extensible programming: Unleashing Julia on GPUs. IEEE Trans. Parallel Distrib. Syst. (2018). arXiv:1712.03112 [cs.PL]. https://doi.org/10.1109/TPDS.2018.2872064
51. Besard, T., Churavy, V., Edelman, A., De Sutter, B.: Rapid software prototyping for heterogeneous and distributed platforms. Adv. Eng. Softw. **132**, 29–46 (2019)

52. Naumov, M., Arsaev, M., Castonguay, P., Cohen, J., Demouth, J., Eaton, J., Layton, S., Markovskiy, N., Reguly, I., Sakharnykh, N., et al.: Amgx: A library for gpu accelerated algebraic multigrid and preconditioned iterative methods. SIAM J. Sci. Comput. **37**(5), 602–626 (2015)
53. Badia, S., Martín, A.F., Verdugo, F.: Gridapdistributed: A massively parallel finite element toolbox in julia. J. Open Source Softw. **7**(74), 4157 (2022)
54. Walker, P.J., Yew, H.-W., Riedemann, A.: Clapeyron.jl: An extensible, open-source fluid thermodynamics toolkit. Ind. Eng. Chem. Res. **61**(20), 7130–7153 (2022). https://doi.org/10.1021/acs.iecr.2c00326
55. Yin, Z., Bruer, G., Louboutin, M.: slimgroup/JutulDarcyRules.jl: v0.2.6. Zenodo (2023). https://doi.org/10.5281/ZENODO.8172164. https://zenodo.org/record/8172164
56. Louboutin, M., Yin, Z., Orozco, R., Grady, T.J., Siahkoohi, A., Rizzuti, G., Witte, P.A., Møyner, O., Gorman, G.J., Herrmann, F.J.: Learned multiphysics inversion with differentiable programming and machine learning. Lead. Edge **42**(7), 474–486 (2023)
57. Lie, K.-A., Møyner, O. (eds.): Advanced Modelling with the MATLAB Reservoir Simulation Toolbox. Cambridge University Press, Cambridge, United Kingdom (2021)

Authors and Affiliations

Olav Møyner[1]

✉ Olav Møyner
olav.moyner@sintef.no

[1] Mathematics & Cybernetics, SINTEF Digital, P.O. Box 124 Blindern, N-0314 Oslo, Norway